

A topological data analysis approach for detecting data poisoning attacks against machine learning based network intrusion detection systems

Galamo F. Monkam^a, Michael J. De Lucia^b, Nathaniel D. Bastian^{a,*}

^a Army Cyber Institute, Department of Electrical Engineering & Computer Science, United States Military Academy, West Point, NY, 10996, United States

^b Army Research Laboratory, U.S. Army Combat Capabilities Development Command, Aberdeen Proving Ground, MD, 21005, United States

ARTICLE INFO

Keywords:

Machine learning security
Data poisoning detection
Topological data analysis
Unsupervised learning
Network security

ABSTRACT

Data poisoning attacks pose a significant security risk to network security software that utilizes machine learning (ML) for network intrusion detection. As network traffic continues to surge, ML becomes indispensable in detecting and characterizing malicious actors attempting to infiltrate computer networks. However, conventional ML assumes a benign environment, leaving room for adversaries to violate this assumption during the training phase. Detecting data poisoning attacks proves to be a challenging task, as attackers employ subtle alterations in the training data to create backdoors, trojans or triggers. Traditional techniques for addressing data poisoning attacks often focus only on enhancing ML model robustness rather than detecting poisoned data, necessitating the development of novel, more effective approaches. Hence, there is an urgent need to develop new methods for identifying poisoned data, ensuring the security of ML. We introduce a novel approach that harnesses the power of topological data analysis and unsupervised learning, enabling the early identification of poisoned data before training an ML model for network intrusion detection. Leveraging our approach, the extraction of topological features and subsequent application of clustering techniques leads to the creation of new clusters exclusively composed of poisoned data for removal prior to ML model training.

1. Introduction

Network security is a critical consideration in military communications, serving as a vital component of having secure devices, sensors and systems within the broader Internet of Battlefield Things (Suri et al., 2016; Azmoodeh et al., 2018). Machine learning has grown in popularity, and organizations have turned to integrating machine learning models into their network security architecture, particularly in network intrusion detection systems (NIDS). In an effort to increase the accuracy and effectiveness of detecting network intrusions, NIDS have been supplemented with ML models to reduce the workload of the network analysts' and to offer insight and ability that may be beyond human capability (Dalvi et al., 2004). However, by incorporating ML models into NIDS, this has opened the door to novel adversarial machine learning attacks that take advantage of the design and implementation of ML models (Alhajjar et al., 2021). Adversarial machine learning (AML) is classified by the National Institute of Science and Technology (NIST) as being concerned with the design of ML algorithms that can resist security challenges, the study of the capabilities of attackers, and the understanding of attack consequences (Tabassi et al., 2019). NIST expands this explanation to categorizing AML attacks as

poisoning attacks and evasion attacks. Poisoning attacks aim to increase the number of misclassified samples at test time by injecting a small fraction of carefully designed adversarial samples into the training data; whereas evasion attacks are where the attacker manipulates input samples to evade (cause a misclassification) a trained classifier at test/deployment time (Tabassi et al., 2019; Shipp et al., 2020). In other words, the poisoning attacks are conducted during the developmental phase of an ML model, and evasion attacks are executed during the deployment phase of a trained ML model. Both of these attack types can greatly degrade model performance; therefore, impacting the NIDS in terms of the assurance. Poisoning attacks can also take many forms. In poisoning, also known as causative attacks, the data or model are altered indirectly or directly. In indirect poisoning, adversaries without access to pre-processed data used by the target model must instead poison the data before pre-processing. In direct poisoning, the data are altered by data injection or data manipulation, or the model is altered directly by logic corruption. In data injection, adversarial inputs are inserted into the original training data, thereby changing the underlying data distribution without changing the features or labels of the original training data. Injected adversarial samples can be optimized by linear

* Corresponding author.

E-mail addresses: galamo.monkam@westpoint.edu (G.F. Monkam), michael.j.delucia2.civ@army.mil (M.J. De Lucia), nathaniel.bastian@westpoint.edu (N.D. Bastian).

<https://doi.org/10.1016/j.cose.2024.103929>

Received 19 November 2023; Received in revised form 2 March 2024; Accepted 29 May 2024

Available online 31 May 2024

0167-4048/Published by Elsevier Ltd.

programming methods that shift the decision boundary of a centroid model (in unsupervised learning), or by gradient ascent on the test error of the model to degrade classification accuracy (in supervised learning). Data manipulation involves adversarial modification of output labels (label manipulation) and input data (input manipulation) of the original training data. Logic corruption is accomplished by an adversary who can tamper with the ML algorithm and thereby alter the learning process and model itself (Tabassi et al., 2019; Wang et al., 2022; Severi et al., 2021). Backdoor attacks are a form of poisoning attacks where an adversary injects stamped malicious data into a training data set in order to trigger the model a certain way during training and give them their desired model evasion effects at model testing time. This can be done with clean-label attacks where the attacker changes the data in the feature vectors but does not have the ability to change the data samples output label—or this can be done with corrupted-label attacks where the attacker has full control of the data and can change the data in the feature vectors as well as the output label (Guo et al., 2022). NIDS are especially susceptible to backdoor attacks because the developer of the NIDS model does not fully control data collection and manipulation (Shipp et al., 2020). NIDS model developers and implementers do not dictate the limits of the network traffic but instead monitor what is already occurring. Users on the network control what data is produced; thus, they dictate the training set for a NIDS on that network. This lack of control over data production gives adversaries the opportunity to distort the training data set to significantly decrease overall performance in the deployment phase.

In this paper, we present several significant contributions. Firstly, we provide a comprehensive overview of the existing approaches employed for detecting data poisoning attacks on ML models. Moreover, we introduce a novel method to identify poisoned data even before training an ML model for network intrusion detection tasks. Our approach showcases an innovative methodology that leverages topological data analysis (TDA) for topological feature extraction and employs unsupervised learning techniques in anomaly detection. This methodology enables us to preprocess network traffic data and revolutionize the detection of data poisoning. By applying TDA to extract topological features, we have the capability to capture intricate topological properties and underlying patterns within the network traffic data. Consequently, we can effectively identify even the most subtle adversarial modifications that may serve as indications of data poisoning attacks. Additionally, we conduct extensive computational experiments by integrating TDA for feature extraction with four distinct unsupervised clustering learning algorithms.

This combined approach proves to be highly effective in detecting poisoned data, specifically adversarial examples generated by two different attack methods. By incorporating TDA into our methodology, we push the boundaries of data poisoning detection, as it unveils complex topological structures and reveals hidden insights that traditional methods fail to capture.

2. Related works

This section is structured into four distinct parts. In the first part, we present an extensive overview and categorization of AML defenses. These defenses are derived from recent publications that comprehensively survey the existing literature in this field. By synthesizing and categorizing these research findings, our aim is to offer readers a comprehensive understanding of the diverse defense strategies employed in the realm of AML. The second part of this section focuses on reviewing relevant works related to data poisoning detection (DPD). We delve into existing literature that explores the detection of data poisoning attacks and examine innovative techniques in this area. Additionally, the third part of this section addresses the utilization of TDA in the context of AML defenses. We explore how TDA can serve as a powerful tool to enhance defense mechanisms against adversarial attacks. Finally, the last part provides a detailed comparison between our proposed approach

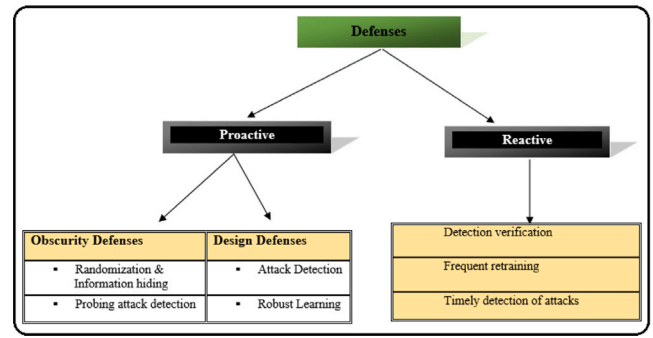


Fig. 1. Categorization of the AML defense techniques.

and existing methods found in the literature for detecting data poisoning attacks against machine learning-based network intrusion detection systems. By critically examining these works, we aim to contribute to the advancement of detection techniques for data poisoning and the exploration of TDA's potential in AML defense strategies.

2.1. Categorization of AML defenses

The deployment of NIDS has significantly increased to detect adversarial activities within networks (Li et al., 2023). However, ML models, which are becoming integral components of NIDS technology, have become susceptible to adversarial examples (De Lucia and Cotton, 2019). Adversarial examples refer to inputs that closely resemble natural data but are misclassified by the ML model. Recent studies indicate that deep neural networks, in particular, are vulnerable to such attacks, suggesting that adversarial attacks might be an inherent weakness of deep learning models. Data poisoning attacks also fall under this category. Given the evolving nature of adversarial attacks, defending against them requires not only countering existing attacks but also anticipating future ones. Thus, robust defenses against adversarial examples are critical and necessitate careful evaluation during the design phase. However, detecting such attacks is challenging since defenders lack knowledge of the triggers, which can be arbitrary shapes or patterns positioned anywhere within the input. Defense methods against backdoor attacks can be categorized into three types. The first type involves creating a filter for the model that can identify abnormal inputs and prevent the activation of backdoors (Gao et al., 2019a; Qi et al., 2020). While this defense can detect backdoors, it cannot remove those hidden within the model. The second type of defense relies on detecting and removing backdoors with the aid of trusted clean data, enabling reverse engineering of backdoor triggers and mitigating their impact. In Venkatesan et al. (2021), a data sanitization method using nested ensemble training method is proposed to mitigate poisoning attacks in NIDS. The third type of defense assumes access to the victim model and the training dataset contaminated by poisoning data. The defender's goal is to sanitize the training dataset, filtering out poisoned data without relying on trusted data. This process allows the creation of a new model through retraining with the sanitized dataset, effectively alleviating the backdoor attack. For instance, Chen et al. propose an activation clustering (AC) method that clusters the activation of neurons in samples to distinguish poisoning samples from the training dataset. In addition to these defense methods employed during training, various adversarial defense techniques have been developed to enhance the resilience of neural networks. To gain a deeper understanding of the inherent properties of adversarial examples and distinguish them from properties influenced by existing attack techniques, we primarily refer to the following sources (Akhtar and Mian, 2018; Biggio and Roli, 2018a; Alhajjar et al., 2021; Devine and Bastian, 2021; Talty et al., 2021; Chakraborty et al., 2018; Liu et al., 2018; Papernot et al., 2018b; Kuznetsov et al., 2018; Lin et al., 2013; Goodfellow et al.,

2018; Yuan et al., 2019; Papernot et al., 2016; Tabassi et al., 2019; Huang et al., 2011; Barreno et al., 2010, 2006). In addition, various sources (Gong et al., 2017; Grosse et al., 2017; Metzen et al., 2017; Feinman et al., 2017; Bhagoji et al., 2017) contribute to constructing defenses aimed at detecting adversarial examples. Two main categories of defenses are proactive defense, which involves preparing effective defense techniques by evaluating potential threats, and reactive defense, which entails immediate defensive actions upon detecting an attack. Reactive defenses aim to counter past attacks, such as analyzing the target classifier, timely detection of novel attacks, frequent classifier retraining, or verifying the consistency of classifier decisions (Lin et al., 2013). On the other hand, proactive defenses focus on preventing future attacks. For a detailed categorization of defenses, please refer to Fig. 1.

2.2. Detection methods for data poisoning attacks

Detecting data poisoning attacks has emerged as a formidable challenge for the ML and cybersecurity communities, leading researchers to propose diverse approaches primarily focused on anomaly or outlier detection. Among these approaches, Principal Component Analysis (PCA) has gained significant popularity as a data preprocessing technique in data poisoning detection (Guo et al., 2018). Researchers have successfully utilized PCA to preprocess various data types, including network traffic data, aiming to enhance the performance of machine learning-based methods in detecting data poisoning attacks. However, PCA has its limitations, especially when applied to high-dimensional data, as it may struggle to capture the underlying data structure and yield suboptimal results in anomaly detection (Romano et al., 2014). Anomaly detection is a prevalent approach for detecting data poisoning, which involves identifying data points that significantly deviate from the expected distribution of the data. The objective is to pinpoint unusual or suspicious data instances that may indicate a data poisoning attack. By monitoring the data's distribution, one can detect any anomalous patterns or outliers that potentially signify the presence of a data poisoning attack. Anomaly detection techniques can be implemented using diverse statistical and machine learning methods, such as Z-score, Mahalanobis distance, or neural networks. Model-based detection techniques have emerged as a powerful approach for detecting data poisoning attacks, where an attacker intentionally manipulates training data to mislead machine learning models and yield inaccurate results (Smith et al., 2020). By training a model on a pristine dataset and subsequently evaluating a test dataset, model-based detection aims to uncover signs of data poisoning. Mathematical models are employed to analyze the training data and identify anomalies that may indicate an attack. Various mathematical models can be employed in model-based detection approaches to scrutinize the training data and detect potential anomalies. For instance, a model can be utilized to identify instances within the training data that deviate significantly from the majority of the samples (Gao et al., 2019b). Another approach involves monitoring changes in the distribution of the training data over time, which can serve as an indication of intentional data manipulation. However, developing a robust model-based detection system for data poisoning attacks poses significant challenges, as attackers can employ sophisticated techniques that are hard to detect (Shu et al., 2022). To overcome these challenges, it is crucial to employ a combination of different detection techniques and continuously monitor and update the detection system to adapt to emerging attack strategies. By integrating multiple approaches and employing vigilant system maintenance, the detection system can enhance its effectiveness in identifying data poisoning attacks.

2.3. Application of topological data analysis in data poisoning detection

- **Topological Signatures for Dataset Representation:** TDA serves as a powerful tool for dataset representation by capturing both the geometric and topological properties inherent within the data. Through

techniques like persistent homology, TDA enables the identification of crucial topological features such as loops, voids, and connected components within the dataset. By utilizing persistent homology to construct a topological signature of the dataset, one can create a concise representation that encapsulates its fundamental structural characteristics. This representation is instrumental in detecting subtle anomalies introduced by poisoning attacks, which have the potential to perturb the dataset's topology in significant yet non-obvious ways. The integration of TDA in this context not only enhances the understanding of the dataset's underlying structure but also facilitates the identification of malicious alterations introduced by adversaries (Carlsson, 2009).

- **Detection of Topological Irregularities:** Poisoning attacks often introduce adversarial perturbations that distort the underlying structure of the data manifold. TDA facilitates the detection of such irregularities by analyzing changes in the dataset's topology induced by these perturbations. By monitoring the persistence of topological features across different scales, algorithms can identify regions of the data where the topology deviates significantly from the expected norm. This enables the pinpointing of potential poisoning instances that disrupt the inherent geometric coherence of the dataset (Hofer et al., 2017).

- **Integration with Machine Learning Models:** Integrating topological features derived from TDA into machine learning models enhances the robustness of DPD systems. By incorporating topological signatures as additional input features, classifiers can leverage both geometric and topological information for improved detection performance. This fusion of TDA with machine learning enables the development of more sophisticated detection frameworks capable of capturing the subtle nuances of poisoning attacks that may evade traditional detection methods (Reininghaus et al., 2014).

- **Enhancing Defense Mechanisms:** Leveraging TDA in DPD contributes to the advancement of defense mechanisms against adversarial attacks. By leveraging topological insights, researchers can develop more comprehensive and resilient defense strategies that complement existing detection techniques. TDA-based approaches offer a holistic perspective on data integrity, enabling the identification of poisoning attacks across diverse datasets and machine learning models (Nicolae et al., 2018).

Incorporating TDA into DPD not only expands the methodological toolkit for detecting poisoning attacks but also underscores the potential of topological insights to fortify the security and reliability of machine learning systems.

2.4. Comparison analysis

In this subsection, we conduct a detailed comparison between our proposed approach and existing methods found in the literature for detecting data poisoning attacks against ML-based NIDS. We aim to elucidate the unique features and advantages of our approach over others. Traditional approaches for detecting poisoned data often encounter challenges in identifying subtle anomalies or early indicators of data poisoning. Furthermore, many existing methods exhibit limited robustness against evolving attack strategies and may suffer from scalability issues when handling large and intricate network datasets. Additionally, while post-training data cleansing methods are commonly employed, they may not adequately mitigate the presence of poisoned data before model training, thereby leaving potential vulnerabilities in the resulting machine learning models. The novelty of our approach resides in the preprocessing pipeline utilized for extracting topological features from network traffic data. While prior research has examined the application of topological data analysis for anomaly detection in various domains, including dynamic multilayer blockchain networks, its specific utilization for detecting data poisoning within network traffic is unprecedented. Our study aims to fill these voids by introducing an innovative methodology that combines TDA with unsupervised learning. By capitalizing on the capabilities of topological features and integrating unsupervised learning algorithms, our methodology

Table 1
Comparison of proposed approach with other proposals.

Aspect	Our approach	Other proposals in literature with citations
Methodology	Utilizes topological data analysis and unsupervised learning	Primarily relies on traditional machine learning techniques (Guo et al., 2020).
Detection Sensitivity	Early identification of poisoned data, maintaining high detection rates	May suffer from delayed detection, compromising system security (Kolias et al., 2017)
Scalability, Interpretability, Adaptability	Suitable for large-scale datasets, provides insights into data structure and attack patterns, and adapts to evolving threats and environments	Performance degradation with increased dataset size, limited interpretability, and manual intervention needed for adaptation (Chen and Lloyd, 2020); (Ribeiro et al., 2016)
Practical Implementation	Feasible implementation with open-source libraries	Implementation complexity and resource requirements may hinder adoption (Papernot et al., 2018a)
Data Removal Strategy	Identifies and removes poisoned data prior to ML model training	May lack explicit data removal strategy. (Biggio and Roli, 2018b). Relies on post-training techniques for detection and mitigation (Campigotto and Poon, 2019)
False Positive Rate	Lower false positive rates in most cases	Higher false positive rates in some cases (Munir et al., 2020)
Transparency	Provides interpretable insights through topological analysis	Lack of transparency in detection mechanisms (Papernot et al., 2017)

facilitates the early identification of poisoned data during the preprocessing phase, thereby mitigating the risk associated with incorporating malicious data into the training process. Additionally, our approach is engineered to adapt to diverse attack patterns and sustain detection sensitivity across a range of network environments, ensuring robust performance amidst evolving threats. Distinguished by its integration of TDA and unsupervised learning, our methodology enables early detection of poisoned data, scalability for large datasets, resilience against varied attack strategies, and efficient removal of tainted data prior to model training. These attributes collectively contribute to superior performance compared to conventional methods for detecting and countering data poisoning attacks targeting NIDS. In Table 1, we present a comprehensive comparison of our proposed approach with existing proposals in the literature, highlighting various aspects such as methodology, feature extraction, generalization, detection sensitivity, scalability, practical implementation, data removal strategy, false positive rate, and transparency.

3. An overview of topological data analysis

This section provides an in-depth exploration of TDA, elucidating its theoretical underpinnings, fundamental concepts, and advanced methodologies. For a better understanding of the TDA approaches and techniques, additional materials are provided in the Appendix section.

3.1. TDA steps

TDA resides at the crossroads of mathematics, computer science, and statistics. Its purpose is to enhance and captivate the field of data analysis. By harnessing concepts from algebraic topology—a mathematical branch that examines the qualities of shapes and spaces—topological data analysis endeavors to derive valuable insights from intricate data collections. TDA has emerged as a potent instrument for scrutinizing datasets with high-dimensionality, which have become increasingly prevalent in fields like genomics, neuroscience, and image analysis. By representing datasets as topological spaces, TDA unveils elusive features such as clusters, voids, and holes that conventional methods struggle to detect. The applications of TDA span an extensive spectrum, encompassing image and signal processing, machine learning, robotics, and more (Altındış et al., 2021; Giansiracusa et al., 2019; Songdechakraiut and Chung, 2020). This interdisciplinary field amalgamates researchers from diverse backgrounds, pooling their expertise to tackle some of the most arduous challenges in contemporary data

analysis (Sizemore et al., 2019). The core steps of TDA, as outlined in Monkam et al. (2023), can be executed through a variety of TDA tools and software packages like Mapper, Giotto-tda, Ripser, and Gudhi. The selection of an appropriate tool depends on the specific application and the complexity of the data set (Tauzin et al., 2021).

3.2. Some TDA definitions

- **The Betti number**, denoted as β_k , is a mathematical concept in algebraic topology that measures the number of k -dimensional holes in a topological space. It provides valuable information about the shape and structure of the space under consideration. Formally, for a topological space X , the k th Betti number β_k is defined as the rank of the k th homology group $H_k(X)$, which is obtained through homology theory. The homology group $H_k(X)$ captures the algebraic representation of k -dimensional cycles, or loops, in X . In simpler terms, β_k counts the number of k -dimensional “voids” or “empty space” in the space X , where the dimension refers to the number of coordinates required to describe the void. For example, β_0 represents the number of connected components in X , β_1 represents the number of independent loops or tunnels, β_2 represents the number of voids bounded by surfaces, and so on. The Betti numbers provide essential information for TDA as they help in identifying and characterizing topological features.

- **Persistent homology** is a mathematical framework within algebraic topology that allows for analyzing and characterizing topological features in data at different scales. It provides a way to capture and study the evolution of these features as parameters change. Let X be a topological space and consider a family of subspaces $X_0 \subseteq X_1 \subseteq \dots \subseteq X_n$, where each X_i represents a “snapshot” of the space at a specific scale or parameter value. The persistent homology of this family of subspaces is obtained by examining the changes in the homology groups as the subspaces vary. Formally, persistent homology introduces the notion of persistent homology groups, denoted as H_k^ϵ , where k is a non-negative integer representing the dimension of the homology group, and ϵ is a parameter representing the scale or threshold. Persistent homology identifies and tracks the appearance and disappearance of homology classes or cycles across different scales. A homology class is considered “persistent” if it persists across a range of scales, indicating the presence of a robust topological feature. The output of persistent homology analysis is often visualized using a persistence diagram or barcode, which represents the birth and death scales of the persistent homology classes. Each point in the diagram corresponds to a persistent

homology class, with the coordinates indicating the scales at which the class is born and dies.

- **A persistence diagram** is a mathematical representation used in persistent homology to visualize the birth and death of topological features across different scales or thresholds. Formally, a persistence diagram is a collection of points in a two-dimensional plane, where each point (b, d) represents a topological feature or homology class that is born at scale b and dies at scale d . The horizontal axis represents the birth scale, and the vertical axis represents the death scale. Persistence diagrams provide a concise and visual summary of the persistence of topological features in a data set. The position and distribution of points in the diagram reveal information about the longevity and significance of these features. Analyzing the persistence diagram helps in understanding the topological structure of the data and extracting meaningful insights.

- **The Wasserstein distance** is a distance measure between two persistence diagrams. Given two persistence diagrams D_1 and D_2 , the p -Wasserstein distance $W_p(D_1, D_2)$ is defined as:

$$W_p(D_1, D_2) = \inf_{\varphi} \left[\sum_{x \in D_1} (|x - \varphi(x)|_q)^p \right]^{1/p} \quad (1)$$

where $\varphi : D_1 \rightarrow D_2$ is a bijection between the points in D_1 and D_2 , $|\cdot|_q$ is the L^q norm, and $p, q \geq 1$. Here, $|x - \varphi(x)|_q$ represents the distance between the x -point in D_1 and its corresponding point in D_2 under the bijection φ . The Wasserstein distance measures the minimum “transportation cost” required to move the mass from one persistence diagram to the other, where the mass is represented by the points in the diagrams. The transportation cost is given by the p th power of the distance between the points, and the minimum total cost is achieved by finding the optimal bijection between the points. Note that if $p = 1$ and $q = \infty$, then the Wasserstein distance is also known as the Earth Mover’s Distance or the Total Variation Distance. The bottleneck distance is another distance measure between two persistent diagrams that are obtained from two different datasets. Given two persistence diagrams D_1 and D_2 , the bottleneck distance $d_B(D_1, D_2)$ is defined as:

$$d_B(D_1, D_2) = \inf_{\phi} \sup_{x \in D_1} |x - \phi(x)|_{\infty} \quad (2)$$

where $\phi : D_1 \rightarrow D_2$ is a bijection between the points in D_1 and D_2 , and $|\cdot|_{\infty}$ is the supremum norm.

- **The bottleneck distance** measures the minimum amount by which we need to “stretch” or “squeeze” the persistence diagrams, such that the furthest pair of points in one diagram is mapped to a pair of points in the other diagram. The distance between these mapped pairs is then the bottleneck distance.

- **The landscape distance** is a distance measure used in TDA to compare two persistence landscapes obtained from two different datasets. Given two persistence Diagram D_1 and D_2 , the landscape distance $d_L(D_1, D_2)$ is defined as:

$$d_L(D_1, D_2) = \left(\sum_{i=1}^k \left(\sum_{j=1}^m |D_{1,i,j} - D_{2,i,j}|^p \right)^{q/p} \right)^{1/q} \quad (3)$$

where k is the number of levels or time intervals, m is the number of functions or “landscapes” at each level, $D_{1,i,j}$ and $D_{2,i,j}$ are the j th function values at the i th level in D_1 and D_2 , respectively, and p and q are parameters that control the sensitivity of the distance measure. A persistence image is a compact representation of the dataset that combines the topological and statistical information of a dataset. It is obtained by applying a function called a filter to the dataset and computing its persistence diagram, which represents the topological features of the dataset as a set of points in a plane. The persistence image is then obtained by mapping each point in the persistence diagram to a pixel in a grid, and assigning a value to the pixel that represents the persistence of the corresponding topological feature. The value assigned to each pixel is computed by integrating a Gaussian

kernel over the region of the persistence diagram that corresponds to the pixel. The persistent image can be used as input to machine learning algorithms for various applications, such as image classification and object recognition.

- **Persistence entropies of persistence diagrams** involve the calculation of entropy measures for sub-diagrams based on distinct homology dimensions (q). To compute the persistence entropy, each sub-diagram is analyzed separately, and their persistence entropies are obtained by calculating the Shannon entropy, using a base 2 logarithm, of the differences $(d - b)$ known as “lifetimes”. These lifetimes represent the duration of the topological features. The calculated entropies are then normalized by dividing them by the sum of all such differences. Additionally, there is an optional normalization step that can be applied based on a simple heuristic (Rucco et al., 2016). Intuitively, this is a measure of the entropy of the points in a persistence diagram. Precisely, let $D = \{(b_i, d_i)\}_{i \in I}$ be a persistence diagram with each $d_i < +\infty$. The persistence entropy of D is defined by:

$$E(D) = - \sum_{i \in I} p_i \log(p_i) \quad (4)$$

where

$$p_i = \frac{(d_i - b_i)}{L_D} \quad \text{and} \quad L_D = \sum_{i \in I} (d_i - b_i)$$

Above, $E(D)$ represents the persistence entropy of the persistence diagram D , p_i represents the probability associated with the i -th point in the diagram, and L_D represents the total length of the persistence diagram D .

- **A filtered simplicial complex** is a sequence of nested simplicial complexes that capture the evolution of topological features in data as a filtration parameter changes. Given two sets of points, S and X , where S is labeled and X is unlabeled, if we define the point set $P = S \cup X$, where $2 \leq q \ll |P|$, a q -simplex $\sigma = [C_0, C_1, \dots, C_q]$ has a dimension $\dim(\sigma) = q$, and it consists of $|\mathbf{V}(\sigma)| = q + 1$ vertices. The set of vertices of σ is denoted as $\mathbf{V}(\sigma) = C_0, C_1, \dots, C_q$. For $0 \leq d \leq q$, a d -simplex τ is called a d -face of σ if $\mathbf{V}(\sigma) \supseteq \mathbf{V}(\tau)$, and σ is called a q -coface of τ . A q -simplex has $\binom{q+1}{d+1}$ d -faces, and the total number of faces is $\sum_{d=0}^q \binom{q+1}{d+1} = 2^{q+1} - 1$. Specifically, 0-faces of a q -simplex σ correspond to individual elements of $\mathbf{V}$, 1-faces are line segments with endpoints in $\mathbf{V}(\sigma)$, 2-faces are triangles with vertices in $\mathbf{V}(\sigma)$, 3-faces are tetrahedra with vertices in $\mathbf{V}(\sigma)$, and so on. This definition provides the necessary elements to build a filtered simplicial complex based on the given point sets and their associated simplices.

4. Dataset introduction and exploration

Here, we introduce the dataset that serves as the basis for the research presented in this paper and explore its characteristics through a comprehensive analysis. UNSW-NB15 and CICIDS2017 are two widely recognized NIDS datasets extensively used in research and evaluation of intrusion detection systems.

The UNSW-NB15 dataset, released by the University of New South Wales in 2015, provides a comprehensive collection of network traffic data. It encompasses both benign and malicious traffic, covering a diverse range of attack scenarios such as DoS, probing, remote-to-local, user-to-root, and infiltration. Captured using tcpdump, the dataset comprises over 2 million labeled network packets, classified into categories including Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms. UNSW-NB15 is an invaluable resource for studying network security, enabling researchers to analyze different attack types and develop effective intrusion detection mechanisms. CICIDS2017 is another prominent dataset widely used for evaluating network intrusion detection systems. Released by the Canadian Institute for Cybersecurity in 2017, it includes both benign and malicious network traffic captured from various network scenarios. With over 2.5 million network flows, the dataset encompasses a wide

Algorithm 1 Pseudocode for Building a Pipeline for TDA 72-Feature Extraction.

```

1: direction_list ← [[0, 1], [1, 0]]
2: center_list ← [[0, 1500], [0, 750], [1500, 0]]
3: filtration_list ← [HeightFiltration(direction, n_jobs=-1) for direction in direction_list] + [RadialFiltration(center, n_jobs=-1) for center in center_list]
4: diagram_steps ← [[Binarizer(threshold=0.4, n_jobs=-1), filtration,
5:                   CubicalPersistence(n_jobs=-1), Scaler(n_jobs=-1)]
6:                   for filtration in filtration_list]
7: metric_list ← [{metric: "bottleneck", metric_params: {}},
8:                {metric: "wasserstein", metric_params: {p: 1}},
9:                {metric: "landscape", metric_params: {p: 1, n_layers: 1, n_bins: 64}},
10:               {metric: "betti", metric_params: {p: 1, n_bins: 64}},
11:               {metric: "heat", metric_params: {p: 1, sigma: 1.6, n_bins: 64}} ]
12: feature_union ← make_union(PersistenceEntropy(nan_fill_value=-1),
13:                             *[Amplitude(metric, n_jobs=-1) for metric in metric_list])
14: tda_union ← make_union(*[make_pipeline(diagram_step, feature_union)
15:                               for diagram_step in diagram_steps], n_jobs=-1)
16: Output: tda_union

```

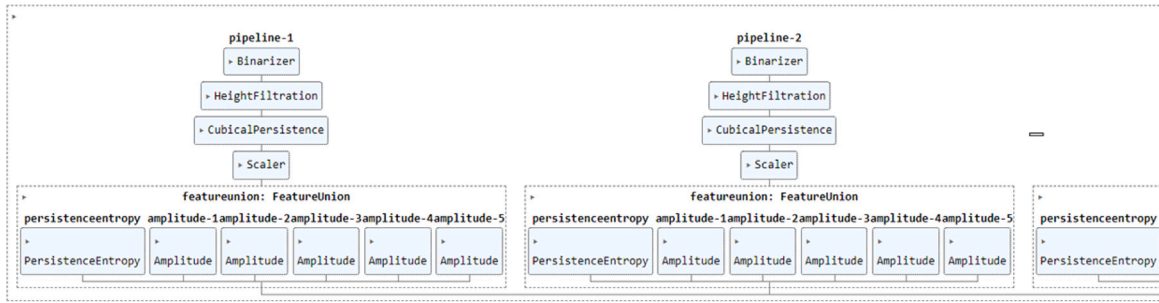


Fig. 2. Diagram of the preprocessing pipeline for TDA feature extraction.

range of attacks, such as DoS, port scans, malware, and botnets. Additionally, CICIDS2017 provides detailed information about the source and destination IP addresses, ports, protocols, timestamps, and flow durations. This dataset serves as a valuable tool for analyzing network traffic patterns, detecting anomalies, and evaluating the effectiveness of intrusion detection algorithms. The breakdown of both datasets can be observed in Figs. 3 and 4, which illustrates the distribution of their respective components. To enhance the analysis, the packet capture data was labeled using Payload-Byte (Farrukh et al., 2022) and transformed into one-dimensional “images” of size 1×1500 (as depicted in Figs. 5 and 6).

Both the UNSW-NB15 and CICIDS2017 data hold significant importance in academic research and industry settings. They serve as valuable resources for the development and evaluation of NIDS. With their well-defined characteristics and widespread adoption, these datasets have emerged as benchmarks for assessing the effectiveness of various approaches employed in network security. Researchers and practitioners alike rely on these datasets to enhance their understanding of network intrusion detection and explore innovative techniques to address evolving security challenges.

5. Methodology

In our innovative approach, we leverage the capabilities of TDA in combination with four unsupervised learning algorithms. We systematically elucidate the complex preprocessing procedures necessary for extracting optimized features, ensuring optimal performance. Additionally, we undertake an insightful examination of network traffic data, demonstrating the effectiveness of our methodology. The four

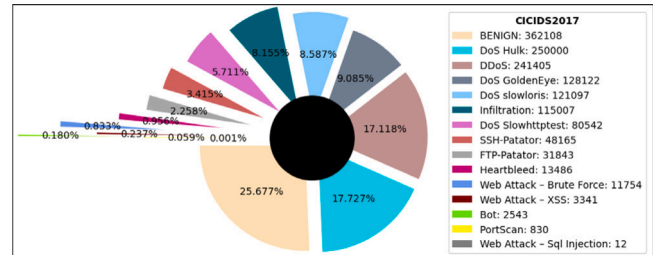


Fig. 3. Composition of CICIDS2017 dataset: Distribution of benign and malicious traffic across attack categories.

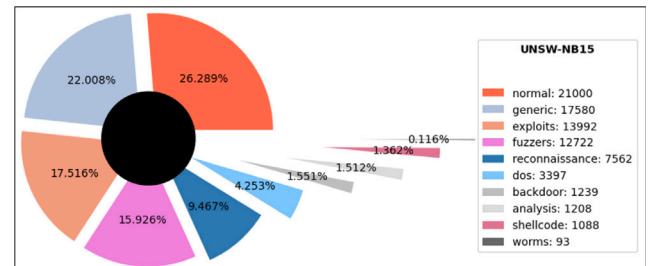


Fig. 4. Composition of UNSW-NB15 dataset: Distribution of benign and malicious traffic across attack categories.

	payload_byte_1	payload_byte_2	payload_byte_3	...	payload_byte_1498	payload_byte_1499	payload_byte_1500
0	66	80	83	...	0	0	0
1	66	80	83	...	0	0	0
2	66	80	83	...	0	0	0
3	66	80	83	...	0	0	0
4	66	80	83	...	0	0	0
...	...	...	...	...	...	...	...

Fig. 5. Snapshot of UNSW-NB15 dataset: Payload-Byte labeled packet capture data, each data point contains 1500 payload bytes with values of 0-255.

	payload_byte_1	payload_byte_2	payload_byte_3	...	payload_byte_1498	payload_byte_1499	payload_byte_1500
0	43	27	1	...	0	0	0
1	23	40	1	...	0	0	0
2	73	58	95	...	0	0	0
3	244	132	1	...	0	0	0
4	124	208	100	...	0	0	0
...	...	...	...	...	...	...	...

Fig. 6. Snapshot of CICIDS2017 dataset: Payload-Byte labeled packet capture data, each data point contains 1500 payload bytes with values of 0-255.

unsupervised algorithms utilized comprise DBSCAN, OPTICS, Mean-Shift Clustering, and HDBSCAN. For a comprehensive understanding of the methodologies and detailed descriptions of these algorithms, we refer the reader to [Appendix A.4](#) of [Appendix](#).

5.1. Rationale for TDA and UL algorithms

Firstly, our decision to employ TDA stemmed from its capability to capture the intricate topological structures inherent in network traffic data. Traditional methodologies often struggle to detect subtle anomalies or early indicators of data poisoning, necessitating a more sophisticated approach. TDA, renowned for its adeptness in analyzing and extracting topological features from high-dimensional datasets, emerges as a compelling solution for identifying such anomalies. By leveraging TDA, our aim is to augment our ability to uncover nuanced patterns and irregularities within network traffic data, thereby bolstering our defense against potential adversarial threats. Furthermore, our selection of unsupervised learning techniques was motivated by the necessity for robustness and adaptability in detecting data poisoning attacks. Unlike supervised methods, unsupervised learning algorithms do not rely on labeled data for training, rendering them well-suited for identifying anomalous patterns in network traffic without prior knowledge of specific attack signatures. This aligns seamlessly with the dynamic and evolving nature of data poisoning attacks, where the attack patterns may shift over time. The combination of TDA with unsupervised learning offers a synergistic advantage in capturing complex data structures and adapting to diverse attack patterns. While TDA excels in analyzing and extracting topological features from high-dimensional datasets, unsupervised learning algorithms provide robustness and adaptability in detecting anomalies within network traffic data. This strategic fusion of methodologies underscores our commitment to deploying cutting-edge techniques for safeguarding against emerging threats in network security. In addition to TDA and unsupervised learning, we meticulously evaluated other potential techniques and algorithms commonly utilized in the realm of network intrusion detection. However, the combination of TDA with unsupervised learning stood out due to its demonstrated effectiveness in capturing complex data structures and its ability to adapt to diverse attack patterns.

5.2. Topological feature extraction

In this section, we unveil our preprocessing pipeline designed for extracting topological features from the CICIDS2017 and UNSW-NB15

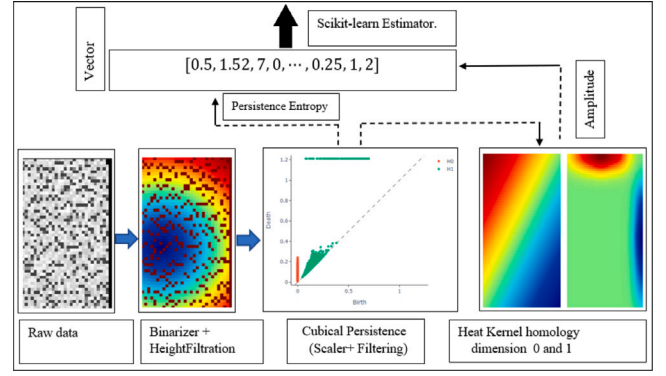


Fig. 7. Pipeline visualization for a single input data.

datasets. By leveraging our pipeline, we process one-dimensional images (1×1500) and witness their transformation into compact representations of size (1×126). Further applying feature reduction techniques, we distill these images down to a streamlined size of (1×72). While we consider both the 126 and 76 feature extractions in this paper, we focus on presenting the algorithm behind the remarkable 72 feature extraction for brevity. For an in-depth understanding of the pre-processing pipeline producing the 126 feature extraction, please refer to [Monkam et al. \(2023\)](#). Algorithm 1 introduces our feature extraction pipeline that harnesses the `gtDA` library ([Tautzin et al., 2021](#)) to compute topological features from input data. Our code orchestrates a series of filtration transformers, persistence diagram generation, and computation of diverse topological metrics. Thanks to the `make_pipeline` and `make_union` functions, we construct a nested pipeline that gracefully applies the same transformations across multiple inputs. The `HeightFiltration` and `RadialFiltration` classes play a pivotal role in creating a collection of filtration transformers, each tailored to specific directions and centers. Within the pipeline, the `diagram_steps` list captures the precise transformations executed on each input image to yield the invaluable persistence diagrams. Sequentially, we utilize the `Binarizer` for thresholding, filtration transformers from the filtration list, the `CubicalPersistence` transformer for diagram generation, and the `Scaler` transformer for optimal diagram normalization. To extract meaningful insights, the `metric_list` outlines a curated selection of topological metrics to be computed on the persistence diagrams. Each metric, represented as a dictionary, encapsulates the metric's name and the associated parameters. Our `feature_union` pipeline applies the `PersistenceEntropy` and `Amplitude` transformers to every persistence diagram. The `PersistenceEntropy` transformer unravels the entropy of the persistence diagram, while the `Amplitude` transformer unlocks the amplitude of each designated topological metric. Subsequently, the `tda_union` pipeline amalgamates the `feature_union` pipeline with the persistence diagrams generated by the filtration transformers, resulting in a harmonized set of topological features meticulously concatenated into a single feature vector for each input. To assess the magnitude of topological features extracted per input, we count the dimensions of each computed feature. The `PersistenceEntropy` transformer contributes a single feature dimension, while the `Amplitude` transformer yields a feature dimension for every specified topological metric. By utilizing the `n_jobs` argument set to `-1`, we optimize data processing by harnessing the full computational power of all available CPU cores on the computer. This strategy ensures rapid computation, particularly beneficial for handling large datasets and intricate feature extraction tasks.

Our presented code encapsulates a sophisticated and robust approach to unveil the intricate topological features latent within data. It harbors immense potential across diverse applications where data shape plays a crucial role. [Fig. 2](#) showcases a Python code snippet that

imports the `set_config` function from the `sklearn` module, configuring the display to a visually appealing diagram format. By invoking the `tda_union` pipeline, we unravel the pipeline's inner workings as an elegant diagram, empowering users with a profound understanding of the TDA feature extraction process and its intricate organizational structure. Fig. 7 offers a visual representation that meticulously outlines the series of steps involved in extracting topological features from a single input data point, following the prescribed methodology detailed in Algorithm 1. This illustrative figure acts as a guiding roadmap, leading us through the critical stages of this intricate process.

Commencing with the raw input data, the algorithm orchestrates a sophisticated sequence of transformations. These transformations encompass several pivotal steps, including Binarization and filtration processes, persistence diagram generation, and the application of heat kernel, alongside the computation of diverse topological metrics. Each of these meticulously crafted steps serves a distinct purpose: capturing various facets of the underlying topological structure inherently embedded within the data. Ultimately, when the code $X_{tda_features} = tda_union.fit_transform(data)$ It marks the beginning of the actual feature extraction process. The pipeline dynamically adjusts to the input data, generating a matrix filled with enlightening topological features. This resulting feature vector becomes a highly valuable resource, enabling the execution of sophisticated ML tasks and unlocking the true potential of the data at-hand.

5.3. Visualization of binarization and filtration

Figs. 8 and 9 offer an illuminating visual representation of a pivotal stage in our TDA process. In these figures, we showcase the application of the Binarization and Filtration techniques to randomly selected input data from the CICIDS2017 and UNSW-NB15 datasets, respectively. This step serves as the foundation for our entire analytical journey. By applying Binarization and Filtration to these data, we prepare the data for subsequent stages of analysis. Binarization involves the transformation of complex data into binary values, simplifying the information for further processing. Filtration, on the other hand, refines the data by emphasizing specific aspects or features that are relevant to our analysis. The careful selection of random input data samples from different categories within each dataset demonstrates the generalizability and effectiveness of these preprocessing techniques across diverse data subsets. This ensures that our subsequent analysis, whether its classification, clustering, or any other data-driven task, will be robust and reliable, as we have laid a solid groundwork for handling the data effectively.

As we progress, Figs. 10 and 11 extend our analysis to include the generation of persistence diagrams and the computation of heat kernels. These additional layers of examination are crucial in uncovering the subtle, often intricate, topological variations that might exist within the data. It is important to highlight that this dual-sided assessment—comparing the original data to their poisoned counterparts provides a holistic perspective. This approach enables us to identify not only potential security threats or anomalies introduced during data poisoning but also to gain a deeper understanding of the underlying data structure and topology.

5.4. Fusion of TDA and UL in computational experimentation

We now present our combination of TDA and unsupervised learning (UL) to achieve our goal of developing an effective method for detecting poisoned data in network packet data. We begin with collecting network traffic data and transforming it into one-dimensional images (1×1500). Each data point, representing 1500 payload bytes with values ranging from 0 to 255, holds valuable information within the packets, essential for intrusion detection purposes. The conversion process expertly maps payload bytes to pixel intensities using grayscale

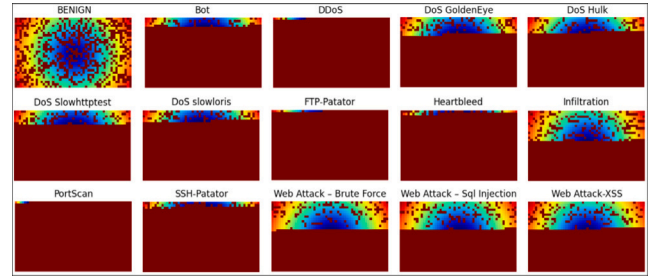


Fig. 8. Binarization and Filtration Visualization on Randomly Selected Data per Categories from the CICIDS2017 Dataset. The (30 × 50) format is used for illustration purposes.

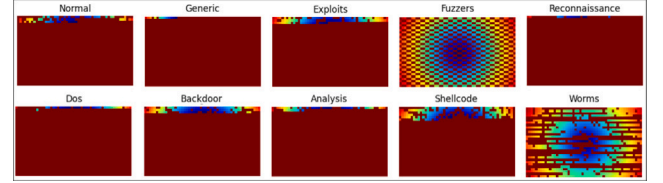
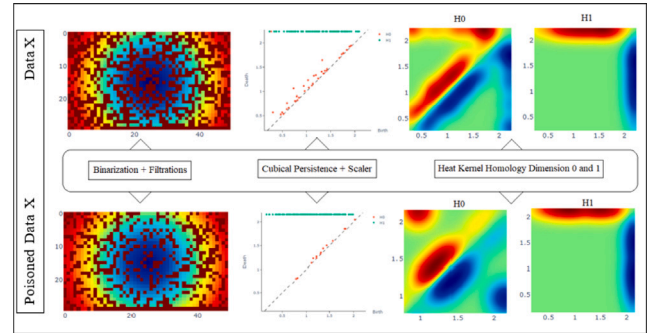


Fig. 9. Binarization and Filtration Visualization on Randomly Selected Data per Categories from the UNSW-NB15 Dataset. The (30 × 50) format is used for illustrative purposes.



employ a genetic algorithm-inspired meta-heuristic and deep reinforcement learning to perturb the payload data, making the malicious packet payload appear benign while retaining packet functionality. Rigorously, the poisoned data also undergoes the same preprocessing pipeline, augmenting the non-poisoned data. We employ the four UL algorithms on both poisoned and non-poisoned data, using identical conditions and hyperparameters. The emergence of new clusters in the poisoned data, compared to the non-poisoned data, becomes a telltale sign of malicious data. Our proposed approach draws strength from a synergy of TDA and UL techniques, offering an effective method for detecting data poisoning attacks within NIDS packet capture data. This powerful combination empowers us to delve deeper into the underlying data structure and patterns, facilitating more effective identification and prevention of data poisoning attacks.

Our decision to represent network traffic data as “images” stems from several critical factors. Firstly, this representation enables us to extract topological features like clusters, holes, and voids through a TDA preprocessing pipeline. By transforming the data into images, our analysis leverages TDA techniques, capturing complex structural patterns and relationships that may signify the presence of poisoned data. Additionally, the image representation enhances the detection of poisoned data by enabling the recognition of visual patterns. Intuitive images effectively convey information through visual cues, allowing both human analysts and machine learning algorithms to visually identify suspicious or anomalous patterns that might suggest the existence of poisoned data. Moreover, the choice of image representation addresses the need for transformation invariance. As network packet payload data may exhibit variations in scale, rotation, or translation, representing the data as images normalizes and accounts for these transformations within the image representation framework. This ensures a robust detection system, capable of effectively handling common data perturbations introduced by attackers.

5.5. Experimentation evaluation metrics

Here, we rigorously assess the performance of four unsupervised learning algorithms applied to two distinct datasets, each categorized into four variations. To illustrate, for a given dataset X , we delve into its pristine form (X), its tampered counterpart (Poisoned X), the outcome of applying the TDA preprocessing pipeline (TDA- X), and the result after poisoning and TDA processing (Poisoned TDA- X), as visually depicted in Figs. 12 through 15. Our primary evaluation criterion for these algorithms is their ability to effectively segregate poisoned data from the original dataset. Thus, our first measure of success is the presence of a distinct cluster comprising solely of poisoned data.

Our second criterion delves into the distribution of poisoned data across these clusters, providing insight into the clustering quality. Additionally, we assess the percentage of data within clusters devoid of poisoned data, which can be a vital criterion, particularly in real-world scenarios where isolating untainted data is crucial. Further, we apply our TDA preprocessing pipeline to both datasets, reducing the feature dimensions from 1500 to 72 and 126, respectively. This reduction leads to our next evaluation criterion: comparing the impact of our unsupervised learning algorithms on datasets with differing feature counts (1500, 126, and 72). This analysis allows us to gain insights into the effectiveness of our TDA preprocessing pipeline in feature reduction and its influence on algorithm performance.

In summary, this section outlines the comprehensive evaluation framework used to gauge the performance of our unsupervised learning algorithms in various scenarios, shedding light on their ability to isolate poisoned data and their resilience to feature dimensionality changes introduced by our TDA preprocessing pipeline.

6. Experimentation results and analysis

The results and analysis presented in this section are pivotal for understanding the effectiveness and performance of our proposed approach in detecting poisoned data within ML-based NIDS. Section 6.1 discusses the process of extracting sensitive information from the dataset before analysis using UL algorithms. This initial step is crucial for ensuring that the data fed into the algorithms is appropriately preprocessed to enhance the accuracy of detection. The second part of this section (6.2 to 6.7) elaborates on the outcomes of applying UL algorithms to both the UNSW-NB15 and CICIDS2017 datasets. Figs. 12 through 15 showcase the results obtained from the application of four distinct UL algorithms, revealing the emergence of distinct clusters with unique characteristics. These clusters provide valuable insights into the composition of poisoned and non-poisoned data instances, facilitating a deeper understanding of the underlying patterns within the datasets. Furthermore, Figs. 16 through 19 delve deeper into the performance of UL algorithms, particularly in detecting red and green clusters. This detailed analysis offers further insights into the core aspects of our research, shedding light on the algorithms' abilities to differentiate between different types of data clusters. Overall, these findings are crucial for validating the efficacy of our proposed approach and highlighting its potential in effectively detecting and mitigating data poisoning attacks against ML-based NIDS.

6.1. TDA feature extraction from network datasets

As shown in Algorithm 1, we employ a systematic approach to extract 72 distinct features from each input data point within both the CICIDS2017 and UNSW-NB15 datasets. For a more comprehensive feature set of 126 per input data, we direct interested readers to consult the detailed work in Monkam et al. (2023). Our method relies on advanced TDA techniques, facilitating the extraction of these 72 features from each input data point. This meticulous process unfolds through a series of well-defined steps.

First, we initiate the process with **binarization**, where the input image is transformed into a binary representation using a threshold value of 0.3, effectively classifying each pixel into either 0 or 1. Following this initial transformation, the binary image undergoes a sequence of **filtration transformers**, including the HeightFiltration and RadialFiltration techniques. While HeightFiltration captures features in vertical and horizontal directions, RadialFiltration explores diverse directions from specific centers.

As the binary image advances, it encounters the CubicalPersistence transformer. This step generates a **persistence diagram** visually representing the image's topological characteristics as distinct points on a diagram. However, there is more to uncover. We delve into Metric Calculation, where we compute eight diverse TDA metrics, such as bottleneck, Wasserstein, landscape, Betti, and heat metrics. These metrics provide scalar values that unveil distinctive features of the input image. The process culminates with the PersistenceEntropy and Amplitude transformers. These transformers extract essential information from the scalar values. PersistenceEntropy reveals the entropy of the persistence diagram, while Amplitude transformers extract the amplitude of each metric. These transformations yield a set of values, each associated with a subdiagram, in accordance with the homology dimension.

The result is a vector of amplitudes, represented as $a = (a_{q_1}, \dots, a_{q_n})$, where q_i spans the homology dimensions. The final output, whether the vector a itself or its norm, depends on the parameter “order”. This rigorous methodology allows us to uncover latent topological features within our data, enhancing our understanding and insights.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) DPDC=Distribution of Poisoned Data Per Cluster														
UNSW-NB15								CICIDS2017						
Clusters	Ud1	Ud2						Clusters	Cd1	Cd2	Cd3			
	43.94%	56.06%							45.07%	37.46%	17.47%			
POISONED UNSW-NB15								POISONED CICIDS2017						
Clusters	Ud3	Ud4						Clusters	Cd4	Cd5	Cd6	Cd7		
	45.04%	54.96%							54.65%	33.54%	5.38%	6.43%		
DPDC	86.45%	13.55%						DPDC	67.4%	12.60%	3.8%	16.20%		
126 features extraction per each input data point								126 features extraction per each input data point						
TDA-UNSW-NB15								TDA-CICIDS2017						
Clusters	Ud5	Ud6	Ud7	Ud8	Ud9	Ud10		Clusters	Cd8	Cd9	Cd10	Cd11	Cd12	
	47.59%	12.50%	9.60%	11.99%	10.93%	7.39%			64.69%	5.64%	8.16%	12.13%	9.38%	
POISONED TDA-UNSW-NB15								POISONED TDA-CICIDS2017						
Clusters	Ud11	Ud12	Ud13	Ud14	Ud15	Ud16	Ud17	Clusters	Cd13	Cd14	Cd15	Cd16	Cd17	Cd18
	49.85%	10.79%	8.42%	11.3%	9.96%	5.69%	3.99%		64.78%	6.19%	7.34%	10.92%	7.74%	3.03%
DPDC	60.10%	0%	0%	0%	0%	0%	39.90%	DPDC	52.37%	11.10%	7.23%	0%	0%	29.30%
72 features extraction per each input data point								72 features extraction per each input data point						
TDA-UNSW-NB15								TDA- CICIDS2017						
Clusters	Ud18	Ud19	Ud20	Ud21	Ud22			Clusters	Cd19	Cd20	Cd21	Cd22		
	41.73%	15.35%	12.89%	11.69%	18.34%				66.91%	7.24%	12.16%	13.69%		
POISONED TDA-UNSW-NB15								POISONED TDA-CICIDS2017						
Clusters	Ud23	Ud24	Ud25	Ud26	Ud27	Ud28	Ud29	Clusters	Cd23	Cd24	Cd25	Cd26	Cd27	Cd28
	43.02%	13.22%	11.10%	10.07%	15.79%	1.94%	4.86%		65.13%	7.63%	10.84%	12.32%	1.28%	2.80%
DPDC	51.4%	0%	0%	0%	0%	0%	48.6%	DPDC	49.10%	10.70%	0%	0%	12.20%	28%

Fig. 12. Statistical analysis: Applying DBSCAN to UNSW-NB15 and CICIDS2017 datasets.

6.2. Unsupervised learning algorithms on network datasets: A general observation

Figs. 12 to 15 depict the results obtained from the application of four distinct unsupervised learning algorithms to both the UNSW-NB15 and CICIDS2017 datasets. These results reveal the emergence of distinct clusters, each exhibiting unique characteristics. Notably, the observed clusters display variations in their composition of poisoned and non-poisoned data. Yellow clusters are observed to contain a heterogeneous mixture of both poisoned and non-poisoned data instances. In contrast, green clusters exclusively contain non-poisoned data points. On the other hand, red clusters are characterized by containing solely poisoned data instances. Pink clusters, which are a subset of the yellow clusters, are predominantly composed of poisoned data, highlighting the intricate dynamics present within the datasets. Furthermore, blue clusters, identified as a derivative of the yellow clusters, encompass a combination of both non-poisoned and entirely poisoned data subsets, indicating the need for detailed examination and analysis. In this section, we elucidate the commonalities discerned in the outcomes derived from applying these unsupervised machine learning algorithms to the aforementioned datasets, thereby illuminating emergent patterns of significance. Upon subjecting all four unsupervised learning algorithms to the Poisoned UNSW-NB15 and CICIDS2017 datasets each characterized by a substantial feature set of 1500, and devoid of TDA-based preprocessing, an intriguing observation emerged: the absence of clusters formed exclusively from poisoned data. This observation underscores the challenge faced by these algorithms in distinguishing between non-poisoned and poisoned data in the absence of TDA preprocessing, leading to each cluster inevitably containing tainted elements. Consequently, the outcome reflects a notable challenge in achieving an accurate detection of data poisoning, with no discernible differentiation observed. However, a contrasting narrative emerges upon empowering these same unsupervised learning algorithms with TDA-based preprocessing and applying them to the Poisoned TDA UNSW-NB15 and TDA-CICIDS2017 datasets, characterized by a more concise feature set of 72 and 126, respectively. In this context, clusters materialize exclusively comprised of poisoned data, as depicted in the visually representations found in Figs. 12 through 15. These clusters serve as a testament to the efficacy of TDA-based preprocessing, enabling the algorithms to pinpoint and segregate poisoned data with unparalleled precision. Moreover, clusters exclusively housing non-poisoned data underscore the potential for robust detection and differentiation enabled by meticulous preprocessing techniques.

6.3. DBSCAN on UNSW-NB15 and CICIDS2017

We apply DBSCAN to two distinct datasets, namely the UNSW-NB15 and the CICIDS2017, to explore the clustering behavior of tainted (poisoned) and non-poisoned data. Surprisingly, our analysis reveals intriguing patterns, shedding light on the composition of these clusters and their significance in identifying malicious data. We also investigate the impact of feature extraction on the clustering results.

When we apply DBSCAN to the POISONED UNSW-NB15 dataset, a fascinating pattern emerges: all the clusters formed are categorized as ‘Yellow’, as visually represented in Fig. 12. These clusters comprise a mix of both tainted (poisoned) and non-poisoned data. Surprisingly, when we apply DBSCAN to the ‘Poisoned TDA-UNSW-NB15’ dataset, a more intricate landscape unfolds. Three primary clusters emerge, color-coded as ‘Yellow’, ‘Red’, and ‘Green’. These clusters exhibit a composition that includes both tainted and untainted data instances. Notably, when we break down the composition of these clusters across the 72 and 126 feature extraction dimensions, we find that they account on average for 46.43%, 49.14%, and 4.43% of the TDA-UNSW-NB15 dataset, respectively. What is remarkable is that approximately 49.14% of the data remains entirely free from any traces of poisoned instances, underscoring its integrity. Meanwhile, the 4.425% cluster exclusively comprises poisoned instances, representing 44.25% of the overall poisoned data population. It is worth emphasizing that the choice of feature extraction dimension significantly impacts DBSCAN’s performance. Notably, it demonstrates superior performance when utilizing the 72-feature extraction method compared to the 126-feature extraction method.

Our preprocessing pipeline yields fascinating insights into the effect of feature extraction. When extracting 72 features per input data point, 52.12% of the data remains untainted, in contrast to 46.16% when employing 126 features. A similar trend is observed with the “Red” clusters. With 72-feature extraction, 4.86% of the data consists exclusively of poisoned data, constituting a substantial 48.60% of the malicious data population. Conversely, with 126 features extraction, only 3.99% of the entire dataset is exclusively composed of malicious data, representing 39.9% of the entire population of poisoned datasets.

When we initially apply DBSCAN to the Poisoned CICIDS2017 dataset, only Yellow clusters were detected. However, when we apply DBSCAN to the Poisoned TDA-CICIDS2017 dataset, we uncover four distinct cluster types: Yellow, Green, Red, and Pink, representing an average distribution of 75.535%, 20.91%, 1.40%, and 2.155% for both the 72-feature and 126-feature extraction methods.

HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise)															
DPDC=Distribution of Poisoned Data Per Cluster															
UNSW-NB15								CICIDS2017							
Clusters	Uh1	Uh2	Uh3	Uh4				Clusters	Ch1	Ch2					
	39.90%	52.17%	4.31%	3.62%					51.78%	48.22%					
POISONED UNSW-NB15								POISONED CICIDS2017							
Clusters	Uh5	Uh6	Uh7	Uh8				Clusters	Ch3	Ch4					
	44.21%	47.28%	4.35%	4.16%					54.85%	45.15%					
DPDC	83%	3.52%	4.48%	9%				DPDC	82.50%	17.50%					
126 features extraction per each input data point								72 features extraction per each input data point							
TDA-UNSW-NB15								TDA-CICIDS2017							
Clusters	Uh9	Uh10	Uh11	Uh12	Uh13	Uh14	Uh15	Clusters	Ch5	Ch6	Ch7	Ch8			
	32.19%	11.63%	8.84%	8.64%	13.19%	4.68%	11.92%		32.81%	23.79%	13.48%	29.92%			
POISONED TDA-UNSW-NB15								POISONED TDA-CICIDS2017							
Clusters	Uh17	Uh18	Uh19	Uh20	Uh21	Uh22	Uh23	Clusters	Ch9	Ch10	Ch11	Ch12			
	42.13%	4.1%	7.96%	7.78%	11.87%	4.21%	10.73%		31.97%	24.41%	33.05%	10.57%			
DPDC	67.9%	0%	0%	0%	0%	0%	0%	DPDC	0%	0%	0%	100%			
72 features extraction per each input data point								126 features extraction per each input data point							
TDA-UNSW-NB15								TDA-CICIDS2017							
Clusters	Uh26	Uh27	Uh28	Uh29	Uh30			Clusters	Ch13	Ch14	Ch15				
	35.52%	22.70%	9.76%	14.3%	17.72%				37.26%	28.23%	34.51%				
POISONED TDA-UNSW-NB15								POISONED TDA-CICIDS2017							
Clusters	Uh31	Uh32	Uh33	Uh34	Uh35	Uh36		Clusters	Ch16	Ch17	Ch18	Ch19			
	36.76%	20.43%	8.78%	12.87%	15.95%	5.21%			39.60%	25.41%	30.05%	4.94%			
DPDC	47.90%	0%	0%	0%	0%	52.1%		DPDC	50.60%	0%	0%	49.40%			

Fig. 13. Statistical analysis: Applying HDBSCAN to UNSW-NB15 and CICIDS2017 datasets.

OPTICS (Ordering Points to Identify the Clustering Structure)															
DPDC=Distribution of Poisoned Data Per Cluster															
UNSW-NB15								CICIDS2017							
Clusters	Uo1	Uo2	Uo3	Uo4	Uo5			Clusters	Co1	Co2	Co3				
	35.83%	23.83%	11.17%	15%	14.17%				42.44%	43.78%	13.78%				
POISONED UNSW-NB15								POISONED CICIDS2017							
Clusters	Uo6	Uo7	Uo8	Uo9	Uo10			Clusters	Co4	Co5					
	35.57%	23.73%	11.50%	16.45%	12.75%				47.11%	52.89%					
DPDC	33.20%	22.80%	14.50%	24.50%	5%			DPDC	39.10%	52.89%					
126 features extraction per each input data point								126 features extraction per each input data point							
TDA-UNSW-NB15								TDA-CICIDS2017							
Clusters	Uo11	Uo12	Uo13	Uo14	Uo15			Clusters	Co6	Co7	Co8				
	48.36%	7%	10.89%	11.01%	22.74%				63.03%	14.56%	22.41%				
POISONED TDA-UNSW-NB15								POISONED TDA-CICIDS2017							
Clusters	Uo16	Uo17	Uo18	Uo19	Uo20	Uo21	Uo22	Clusters	Co9	Co10	Co11	Co12	Co13		
	52.91%	1.30%	9.80%	9.91%	20.47%	2.58%	3.03%		56.73%	16.30%	21.46%	1.75%	3.76%		
DPDC	43.9%	0%	0%	0%	0%	25.80%	30.30%	DPDC	0%	32%	12.90%	17.5%	37.6%		
72 features extraction per each input data point								72 features extraction per each input data point							
TDA-UNSW-NB15								TDA-CICIDS2017							
Clusters	Uo23	Uo24	Uo25	Uo26	Uo27			Clusters	Co14	Co15	Co16	Co17			
	23.19%	49.63%	3.50%	12%	11.68%				30.55%	22.09%	19.69%	27.67			
POISONED TDA-UNSW-NB15								POISONED TDA-CICIDS2017							
Clusters	Uo28	Uo29	Uo30	Uo31	Uo32	Uo33	Uo34	Clusters	Co18	Co19	Co20	Co21	Co22		
	47.23%	23.87%	10.80%	10.51%	1.18%	0.89%	3.33%		37.80%	25.32%	30.68%	1.19%	5.01%		
DPDC	24.10%	0%	0%	0%	11.8%	8.90%	21.90%	DPDC	25.8%	12.20%	0%	11.90%	50.10%		

Fig. 14. Statistical analysis: Applying OPTICS to UNSW-NB15 and CICIDS2017 datasets.

Notably, 20.91% of the data remains uncontaminated by any traces of poisoned data, while 1.40% exclusively comprises poisoned instances, representing 14% of the poisoned data population. Once again, DBSCAN's performance stands out when utilizing the 72-feature extraction approach, where an impressive 23.16% of the data is entirely free from malicious data, compared to 18.66% when using the 126-feature extraction method. This trend holds true for the "Red" clusters as well, as using the 72-feature extraction approach reveals that 2.8% of the data exclusively comprises poisoned instances, contributing to a substantial 28% of the malicious data population. Conversely, with the 126-feature extraction approach, no Red clusters are observed. In summary, our analysis of DBSCAN on both datasets yields valuable insights into the distribution of tainted and untainted data. The effectiveness of our TDA-preprocessing pipeline for feature extraction is pivotal in identifying exclusively poisoned clusters and untainted clusters, with the 72-feature extraction consistently outperforming the 126-feature approach. These findings are crucial for enhancing data security and threat detection strategies.

6.4. HDBSCAN on UNSW-NB15 and CICIDS2017

When applying HDBSCAN to the POISONED UNSW-NB15 dataset, a striking phenomenon emerges. All the clusters formed are prominently labeled as "Yellow", as indicated in Fig. 13. These clusters

exhibit a unique composition, comprising both tainted (poisoned) and non-poisoned data. Surprisingly, when we extend this analysis to the Poisoned TDA-UNSW-NB15 dataset, we observe the emergence of three distinct cluster types: Yellow, Green, and Red. These clusters correspond to the TDA-UNSW-NB15 dataset components, constituting approximately an average distribution of 36.16%, 59.50%, and 4.34% of the data, respectively. Remarkably, this implies that, on average, 59.50% of the data contains absolutely no traces of poisoned data, while 4.34% is exclusively composed of poisoned data, representing a substantial 43.40% of the entire poisoned data population.

A notable finding is that HDBSCAN performs notably better when employing 72-feature extraction compared to 126-feature extraction. Specifically, with the former, we achieve a higher percentage of data devoid of malicious elements, at 57.98%, compared to 46.16% with the latter. This trend is also evident in the Red clusters, where using 72 features results in 6.8% of the data exclusively composed of poisoned data, representing 68% of the malicious data population. In contrast, with 126 feature extraction, only 3.99% of the entire dataset is exclusively comprised of malicious data, amounting to 39.9% of the overall population of poisoned datasets. Conversely, when we applied HDBSCAN to the Poisoned CICIDS2017 dataset, initially, only Yellow clusters surfaced. However, upon analyzing the POISONED TDA-CICIDS2017 dataset, we discovered five distinct cluster types:

Mean Shift Clustering														
DPDC=Distribution of Poisoned Data Per Cluster														
UNSW-NB15							CICIDS2017							
Clusters	Um1	Um2	Um3	Um4	Um5	Um6	Clusters	Cm1	Cm2	Cm3	Cm4			
	19.25%	24.68%	30.53%	13.44%	6.39%	5.70%		41.13%	19.56%	17.50%	21.81%			
POISONED UNSW-NB15							POISONED CICIDS2017							
Clusters	Um7	Um8	Um9	Um10			Clusters	Cm5	Cm6	Cm7	Cm8			
	21.67%	27.87%	32.61%	17.85%				37.91%	21.63%	18.76%	21.70%			
DPDC	24.60%	35.43%	22.42%	17.55%			DPDC	8.90%	40.30%	30.10%	20.70%			
126 features extraction per each input data point							126 features extraction per each input data point							
TDA-UNSW-NB15							TDA-CICIDS2017							
Clusters	Um11	Um12	Um13				Clusters	Cm9	Cm10	Cm11	Cm12	Cm13		
	47.52%	17.62%	34.86%					22.83%	31.10%	13%	20.28%	12.79%		
POISONED TDA-UNSW-NB15							POISONED TDA-CICIDS2017							
Clusters	Um14	Um15	Um16	Um17			Clusters	Cm14	Cm15	Cm16	Cm17	Cm18	Cm19	Cm20
	31.77%	30.11%	33.30%	4.82%				20.55%	27.99%	14.76%	20.55%	11.51%	1.89%	2.75%
DPDC	0%	32.50%	19.30%	48.20%			DPDC	0%	0%	30.60%	23%	0%	18.90%	27.50%
72 features extraction per each input data point							72 features extraction per each input data point							
TDA-UNSW-NB15							TDA-CICIDS2017							
Clusters	Um18	Um19	Um20	Um21	Um22		Clusters	Cm21	Cm22	Cm23	Cm24			
	21.71%	34.23%	18.72%	15.48%	9.86%			8.01%	32.76%	16.81%	42.42%			
POISONED TDA-UNSW-NB15							POISONED TDA-CICIDS2017							
Clusters	Um23	Um24	Um25	Um26	Um27	Um28	Clusters	Cm25	Cm26	Cm27	Cm28	Cm29	Cm30	
	33.42%	31.96%	18.30%	11.22%	1.13%	3.97%		10.33%	29.40%	15.13%	38.18%	1.68%	5.28%	
DPDC	0%	11.50%	14.50%	23%	11.3%	39.70%	DPDC	30.40%	0%	0%	0%	16.8%	52.80%	

Fig. 15. Statistical analysis: Applying mean shift clustering to UNSW-NB15 and CICIDS2017 datasets.

Yellow, Green, Red, and Blue, constituting, on average (with both 72 and 126 feature extraction), 19.8%, 72.44%, 2.47%, and 5.29% of the data, respectively. Notably, a substantial 72.44% of the data contains no traces of poisoned data, while 2.47% exclusively comprises poisoned instances, representing a significant portion, or 24.7%, of the poisoned data population. In this scenario, the choice between 72 or 126 feature extraction methods warrants careful consideration, as it can be highly situational.

Further investigation reveals that when using 72 features per input data point, only two clusters form: Green and Blue, representing 89.43% and 10.57%, respectively. An intriguing outcome is that 89.43% of the data is entirely free from malicious elements, while the remaining 10.57% contains 100% of the poisoned data population. This result suggests a valuable potential for isolating threats during emergencies, with less than 1% of non-tainted data being compromised. The 89.43% represents an impressive 99.33% of non-tainted data, a result that warrants a more detailed examination. However, when employing the 126-feature extraction method, three clusters emerge: Yellow, Green, and Red, accounting for 39.6%, 55.46%, and 4.94%, respectively. Given our general interest in Green and Red clusters, it is evident that using 72 features extraction yields a higher percentage of non-tainted data, 89.43%, compared to 55.46% obtained with 126 features extraction. Additionally, the 4.94% exclusively comprises poisoned data, representing 49.4% of the entire population of poisoned data.

In sum, our analysis of HDBSCAN on UNSW-NB15 and CICIDS2017 datasets uncovers intriguing clustering patterns that depend on feature extraction methods, offering valuable insights for data security and anomaly detection.

6.5. OPTICS on UNSW-NB15 and CICIDS2017

When OPTICS is applied to the POISONED UNSW-NB15 dataset, a distinctive clustering pattern emerges, where all clusters are consistently labeled as “Yellow”, as highlighted in Fig. 14. These Yellow clusters comprise a mixture of both tainted (poisoned) and untainted data. However, a noteworthy discovery unfolds when OPTICS is employed on the Poisoned TDA-UNSW-NB15 dataset, revealing the presence of four distinct cluster types: Yellow, Green, Red, and Pink, with average proportions of approximately 47.54%, 45.83%, 6.03%, and 0.59%, respectively. Interestingly, on average, about 45.83% of the data contains no traces of poisoned data, while approximately 6.03% exclusively

consists of poisoned data, making up a substantial 60.30% of the poisoned data population. OPTICS demonstrates superior performance with 72 feature extractions compared to the 126 feature extraction method. Specifically, when utilizing 72 features per input data point, approximately 6.03% of the data is exclusively composed of poisoned data, representing a significant 60.3% of the malicious data population. Conversely, with 126 feature extractions, approximately 5.61% of the entire dataset is exclusively composed of malicious data, accounting for 56.1% of the entire population of poisoned datasets.

On the contrary, when OPTICS is initially applied to the Poisoned CICIDS2017 dataset, only Yellow clusters are observed. However, a different scenario unfolds when OPTICS is employed on the POISONED TDA-CICIDS2017 dataset, revealing the presence of three distinct cluster types: Yellow, Green, and Red, with average proportions of approximately 43.71%, 50.44%, and 5.85%, respectively (utilizing both 72 and 126 feature extractions). It is important to note that, on average, approximately 50.44% of the data is entirely devoid of any traces of poisoned data, while approximately 5.85% exclusively consists of poisoned instances, constituting a significant portion of the poisoned data population, amounting to 58.5%. In practice, when our preprocessing pipeline generates 72 features per input data point, OPTICS effectively identifies clusters exclusively comprised of malicious data, accounting for approximately 6.2% of the data, corresponding to 60.3% of the population of poisoned datasets. Similarly, when employing the 126-feature extraction method, three distinct cluster types emerge, encompassing Yellow, Green, and Red Clusters, with average proportions of approximately 37.76%, 56.73%, and 5.51%, respectively. Clearly, utilizing 72 feature extractions is the preferred choice as it results in higher accuracy when distinguishing poisoned data from untainted data.

6.6. Mean shift clustering on UNSW-NB15 and CICIDS2017

In the case of the POISONED UNSW-NB15 dataset, Mean Shift Clustering (MSC) consistently reveals clusters labeled as “Yellow”, as displayed in Fig. 15. These clusters amalgamate both tainted (poisoned) and untainted data. Notably, for the Poisoned TDA-UNSW-NB15 dataset, a remarkable discovery unfolds with the emergence of four distinct cluster types: Yellow, Green, Red, and Pink. On average, these clusters represent proportions of approximately 38.09%, 56.95%, and 4.96%, respectively.

What is truly surprising is that, on average, a significant 56.95% of the data contains no traces of poisoned data, while approximately 4.96% exclusively comprises poisoned data, representing a substantial 49.6% of the poisoned data population. Intriguingly, with 72 feature extractions, around 5.10% of the data is solely composed of poisoned data, constituting 51% of the malicious data population. In contrast, employing 126 feature extractions results in approximately 4.82% of the entire dataset being exclusively composed of malicious data, representing 48.20% of the overall population of poisoned datasets. This showcases that MSC performs more effectively with the 72 feature extraction method compared to the 126 feature extraction method.

Turning our attention to the Poisoned CICIDS2017 dataset, the initial application of MSC exclusively reveals Yellow clusters. However, a different scenario unfolds when MSC is deployed on the POISONED TDA-CICIDS2017 dataset, leading to the emergence of four distinct cluster types: Yellow, Green, Red, and Pink. These clusters, on average (using both 72 and 126 feature extractions), represent proportions of approximately 22.78%, 71.38%, 4.96%, and 0.88%, respectively. It is worth noting that, on average, a substantial 71.38% of the data remains completely free from any traces of poisoned data, while approximately 4.96% exclusively comprises poisoned instances, making up a significant 49.6% of the poisoned data population.

In practice, with our preprocessing pipeline deriving 72 features per input data point, MSC adeptly identifies clusters exclusively formed of malicious data, encompassing approximately 7.04% of the data, which corresponds to a remarkable 70.4% of the poisoned data population. Utilizing the 126-feature extraction method, three distinct cluster types surface, including Yellow, Green, and Red Clusters, with average proportions of approximately 35.31%, 60.04%, and 4.64%, respectively. Evidently, opting for the 72 feature extraction method is highly preferable as it ensures higher accuracy when attempting to isolate poisoned data from untainted data.

6.7. Accuracy of cluster detection in Red and Green categories across both datasets

Figs. 16 to 19 offer valuable insights into the performance of unsupervised learning algorithms in detecting red and green clusters, shedding light on the core aspects of our research. The green clusters signify our UL models' proficiency in identifying untainted data, while the red clusters gauge their effectiveness in uncovering exclusively poisoned data. Both these cluster types carry substantial implications for our study. These figures serve as instrumental tools for evaluating the impact of feature extraction across a diverse range of unsupervised learning clustering algorithms.

Additionally, they provide key insights into which unsupervised learning algorithms excel at detecting both "red" and "green" clusters—both of which hold significant significance. The "green" clusters underscore our unsupervised learning algorithms' capacity to recognize untainted data clusters, highlighting their adeptness in discerning genuine patterns. Conversely, the "red" clusters measure the algorithms' effectiveness in identifying exclusively contaminated or poisoned data. Moreover, Figs. 20 and 21 vividly depict the probability distribution of both red and green clusters within our datasets. These visual aids not only breathe life into our findings but also make them more accessible and engaging for our audience.

Analysis of Figs. 16 and 18 reveals that with 72-feature extraction applied to both datasets, the average detection accuracy of green clusters across all four UL models stands at 46.69% and 62.83%, respectively. However, when using 126 features per input data, the average detection accuracy for both datasets drops to 43.52% and 38.91%, respectively. It is evident that the former feature extraction method outperforms the latter. Fig. 20 provides a comprehensive visual representation of the probability distribution of green clusters across both datasets, making it easier to grasp the distribution trends. Furthermore, an examination of Figs. 17 and 19 indicates that with 72-feature

POISONED TDA-UNSW-NB15 Green Clusters: Poison-Free Data			
	72-Features Extraction	126-Features Extraction	Average 1
DBSCAN	50.18%	46.16%	48.17%
HDBSCAN	57.98%	54.66%	56.32%
OPTICS	45.18%	41.48%	43.33%
MSC	33.42%	31.77%	32.60%
AVERAGE 2	46.69%	43.52%	

Fig. 16. Detection accuracy of the Green Clusters within the poisoned TDA-UNSW-NB15 dataset.

POISONED TDA-UNSW-NB15 Red Clusters: Exclusively Poisoned Data			
	72-Features Extraction	126-Features Extraction	Average 3
DBSCAN	4.86%	3.99%	4.43%
HDBSCAN	5.47%	3.21%	4.34%
OPTICS	6.41%	5.61%	6.01%
MSC	5.1%	4.82%	4.96%
AVERAGE 4	5.46%	4.41%	

Fig. 17. Detection accuracy of the Red Clusters within the poisoned TDA-UNSW-NB15 dataset.

POISONED TDA-CICIDS2017 Green Clusters: Poison-Free Data			
	72-Features Extraction	126-Features Extraction	Average 7
DBSCAN	23.16%	18.66%	20.91%
HDBSCAN	89.43%	55.46%	72.45%
OPTICS	56%	21.46%	38.73%
MSC	82.71%	60.05%	71.38%
AVERAGE 8	62.83%	38.91%	

Fig. 18. Detection accuracy of the Green Clusters within the poisoned TDA-CICIDS2017.

extraction per input data across both datasets, the average detection accuracy of red clusters for all four UL models is 5.46% and 3.99%, respectively. Conversely, when utilizing 126 features per input data, the average detection accuracy across both datasets decreases to 4.41% and 3.77%, respectively. Once again, it is clear that the 72-feature extraction method is more effective. Fig. 21 complements this analysis by offering a comprehensive visual representation of the probability distribution of red clusters across both datasets.

7. Experiment steps for reproducibility

This section provides comprehensive guidance on algorithm configurations and experiment steps essential for ensuring reproducibility in clustering analysis. It encompasses detailed discussions on parameter settings for DBSCAN, HDBSCAN, OPTICS, and Mean-Shift Clustering algorithms. In DBSCAN, the epsilon parameter defines the maximum distance between two samples for them to be considered neighbors, while min_samples specifies the minimum number of samples required

POISONED TDA-CICIDS2017 Red Clusters: Exclusively Poisoned Data			
	72-Features Extraction	126-Features Extraction	Average 5
DBSCAN	2.80%	(0.00%)*	1.40%
HDBSCAN	(0.00%)**	4.94%	2.47%
OPTICS	6.20%	5.51%	5.86%
MSC	6.96%	4.64%	5.80%
AVERAGE 6	3.99%	3.77%	

Fig. 19. Detection accuracy of the Red Clusters within the poisoned TDA-CICIDS2017.

for core point identification. To ensure reproducibility, it is recommended to select epsilon within the range (0, 2] and min_samples within [2, 300]. This flexibility allows for adjusting the neighborhood size and density threshold for effective cluster formation. Similarly, for HDBSCAN, the min_cluster_size parameter plays a pivotal role in cluster formation. A suggested range of [2, 400] for min_cluster_size ensures reproducibility and enables the identification of clusters with varying sizes and densities. In OPTICS, critical parameters include min_samples, epsilon, and min_cluster_size. Setting min_samples within [2, 300], epsilon in the range (0, 2], and min_cluster_size within [2, 400] is recommended for reproducibility. These parameters collectively influence the granularity and size of clusters, with min_samples defining core points and min_cluster_size setting the threshold for minimum cluster size. Additionally, Mean-Shift Clustering, as a non-parametric method, automatically adapts to data distribution without the need for explicit hyperparameter setup. This characteristic makes it particularly appealing for clustering tasks where the underlying data characteristics are unknown or variable. While initial parameter ranges are provided for reproducibility, it is acknowledged that experimentation and iterative refinement may be necessary to identify optimal settings tailored to dataset characteristics and analytical objectives.

8. Discussion and conclusions

Our innovative fusion of TDA and unsupervised learning emerges as a game-changing solution for detecting poisoned data within network traffic. This comprehensive approach offers unparalleled precision and adaptability in safeguarding against data poisoning attacks.

The Crucial Role of TDA Preprocessing: Foremost, our research highlights the pivotal role played by our TDA preprocessing pipeline. It is the linchpin without which all four unsupervised learning algorithms struggled to effectively differentiate between non-poisoned and poisoned data. This struggle manifested in clusters that inevitably contained elements of both tainted and untainted data, underscoring the indispensable nature of TDA preprocessing in fortifying these algorithms against malicious data.

Distinct Cluster Formations: Our observations unveiled the formation of clusters devoid of poisoned data, particularly exemplified by the emergence of “Green” clusters. These clusters represent pristine segments of the dataset, untainted and ready for precise detection and isolation of non-poisoned data segments. Additionally, the discovery of “Red” clusters exclusively harboring poisoned data is of paramount significance, showcasing the algorithms’ remarkable precision in identifying and segregating malicious instances.

The Impact of Feature Extraction: The strategic choice of feature extraction method played a pivotal role in our findings. Our analysis consistently favored the 72 features extraction over the 126 features extraction, underscoring its superiority in identifying both non-poisoned

and poisoned data segments. This revelation not only enhances computational efficiency but also bolsters the performance of unsupervised learning algorithms.

Exploring Further Insights: Beyond our primary observations, our study unearthed nuances in the behavior of these algorithms. Certain algorithms demonstrated the ability to form distinct clusters even in the absence of TDA preprocessing, hinting at variations in their inherent capabilities. These nuances beckon further investigation, potentially leading to algorithm-specific strategies for bolstering cybersecurity.

Future Explorations: In future work we aim to investigate the feasibility of removing the “Red” and “Green” clusters from the entire dataset, followed by iteratively rerunning clustering algorithms on the remaining portion—a mixture of tainted and untainted data. The ultimate goal is to completely eliminate poisoned data from our datasets or arrive at clusters consisting of a minimal blend of both tainted and untainted data.

In conclusion, our research underscores the indispensable role of TDA preprocessing in empowering unsupervised learning algorithms to effectively discern between non-poisoned and poisoned data. The emergence of distinct clusters devoid of poisoned data and the isolation of exclusively poisoned clusters exemplify the potential of these techniques in the realm of cybersecurity. The preference for 72 features extraction streamlines the process, enhancing efficiency. As we continue to navigate the intricate landscape of cybersecurity, these findings are poised to catalyze the development of more robust and precise threat detection systems, ensuring the resilience of our network data security.

Algorithm 2 DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

```

1: procedure DBSCAN( $D, \epsilon, \text{MinPts}$ )
2:   Initialize an empty list of clusters  $C$ 
3:   for each unvisited point  $P$  in dataset  $D$  do
4:     Mark  $P$  as visited
5:      $N \leftarrow \text{Neighbors}(P, \epsilon)$   $\triangleright$  Find points within  $\epsilon$  distance
6:     if  $|N| < \text{MinPts}$  then
7:       Mark  $P$  as noise  $\triangleright$  Point has insufficient neighbors
8:     else
9:       Create a new cluster  $C_i$ 
10:      ExpandCluster( $P, N, C_i, \epsilon, \text{MinPts}$ )
11:      Add  $C_i$  to the list of clusters  $C$ 
12:     end if
13:   end for
14:   return  $C$   $\triangleright$  List of clusters
15: end procedure
16: procedure EXPANDCLUSTER( $P, N, C, \epsilon, \text{MinPts}$ )
17:   Add  $P$  to cluster  $C$ 
18:   for each point  $Q$  in  $N$  do
19:     if  $Q$  is not visited then
20:       Mark  $Q$  as visited
21:        $N' \leftarrow \text{Neighbors}(Q, \epsilon)$   $\triangleright$  Find neighbors of  $Q$ 
22:       if  $|N'| \geq \text{MinPts}$  then
23:          $N \leftarrow N \cup N'$   $\triangleright$  Merge neighbor sets
24:       end if
25:     end if
26:   if  $Q$  is not yet a member of any cluster then
27:     Add  $Q$  to cluster  $C$ 
28:   end if
29: end for
30: end procedure

```



Fig. 20. Probability distribution of Green Clusters in both datasets.

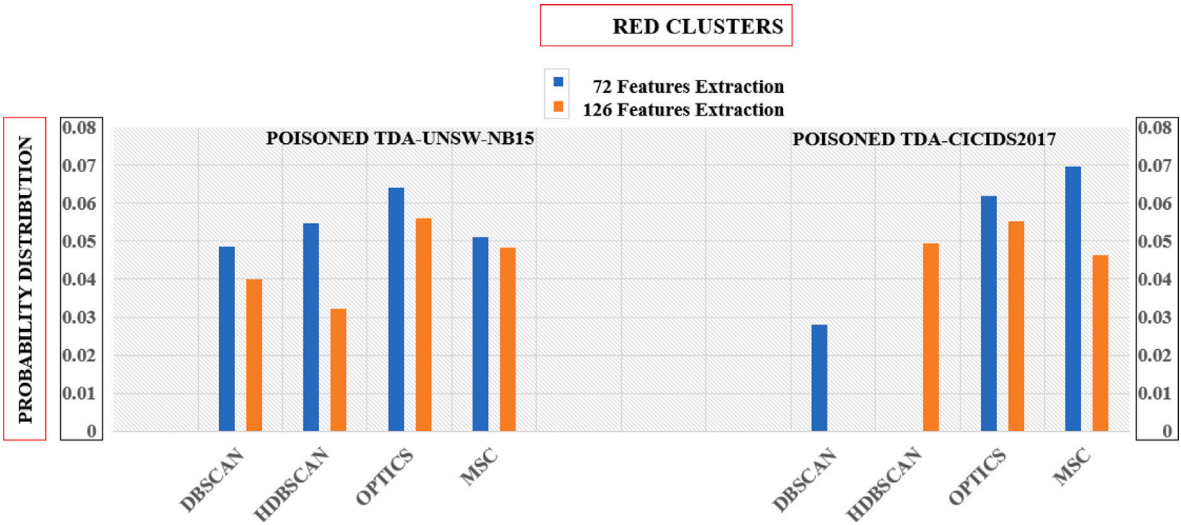


Fig. 21. Probability distribution of Red Clusters in both datasets.

CRediT authorship contribution statement

Galamo F. Monkam: Data curation, Formal analysis, Investigation, Methodology, Visualization, Writing – original draft. **Michael J. De Lucia:** Resources, Writing – review & editing. **Nathaniel D. Bastian:** Conceptualization, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported in part by the U.S. Army Combat Capabilities Development Command (DEVCOM) Army Research Laboratory under Support Agreement No. USMA21050 using computational resources from the Intelligent Cyber-Systems and Analytics Research Lab at the Army Cyber Institute. The views expressed in this paper are those of the authors and do not reflect the official policy or position of the U.S. Military Academy, U.S. Army, U.S. Department of Defense, or U.S. Government.

Algorithm 3 Mean Shift Clustering.

```

1: procedure MEANSHIFT( $D$ , bandwidth)
2:   Initialize an empty list of cluster centroids  $C$ 
3:   for each data point  $x$  in dataset  $D$  do
4:     Initialize the mean shift vector  $m(x) = 0$ 
5:     Initialize a weight  $w(x) = 0$ 
6:     while Convergence not reached do
7:       Compute the mean shift vector  $m(x)$  as the weighted average
       of data points within the bandwidth
8:       Update the data point  $x$  as  $x = x + m(x)$ 
9:       Update the weight  $w(x)$  as the sum of weights of data points
       within the bandwidth
10:    end while
11:    if  $x$  is not close to any existing cluster centroid then
12:      Create a new cluster centroid at  $x$ 
13:    end if
14:  end for
15:  return List of cluster centroids  $C$ 
16: end procedure

```

Appendix**A.1. TDA toolkit for ML and data exploration**

Giotto-tda is an open-source Python library for Topological Data Analysis that offers a user-friendly interface for constructing and analyzing various types of topological representations of data. This library, built on top of the scikit-learn machine learning framework, facilitates seamless integration of TDA into machine learning workflows. Giotto-tda supports diverse input data types, including point clouds, graphs, time series, and images, making it a versatile tool for exploring complex datasets (Tauzin et al., 2021). By leveraging Giotto-tda, researchers and practitioners can harness the power of TDA to uncover hidden structures and patterns within their data, enhancing the interpretability and performance of machine learning models.

A.2. Morse theory

Morse theory provides a mathematical framework for characterizing the topology of a dataset by associating a real-valued function, known as a Morse function, with the data. The critical points of the Morse function correspond to the topological features of the dataset, such as peaks, valleys, saddles, and so forth. Moreover, the flow lines of the Morse function capture the relationships between these critical points, providing insights into the connectivity and topology of the underlying data manifold. By applying Morse theory, researchers can gain a deeper understanding of the geometric and topological properties of their data, facilitating the identification of relevant features and structures for further analysis (Milnor, 1963). This theoretical framework enriches the toolkit of Topological Data Analysis, enabling more nuanced and comprehensive exploration of complex datasets.

A.3. Mapper algorithm

The Mapper algorithm is a powerful tool in TDA for visualizing and analyzing high-dimensional datasets. It constructs a simplified, topological representation of the data by partitioning the dataset into overlapping subsets (bins) based on a chosen clustering or cover scheme. Each bin is then summarized by a representative point, typically computed as the centroid or another statistical measure of the data points within the bin. By connecting these representative points based on their overlap in the high-dimensional space, Mapper generates a graph-like structure, called a Mapper graph, which captures the essential topological

features of the dataset. This graph can be further analyzed using graph-theoretical methods to extract insights into the data's underlying structure and relationships. The Mapper algorithm offers a flexible and scalable approach to exploring complex datasets, making it a valuable tool for data visualization, dimensionality reduction, and exploratory data analysis (Singh et al., 2007).

A.4. Cluster analysis with unsupervised learning algorithms**Algorithm 4** HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise)

```

1: procedure HDBSCAN( $D, \epsilon, \text{MinPts}$ )
2:   Compute the pairwise distance matrix  $D_{ij}$  for all data points
3:   Initialize an empty list of clusters  $C$ 
4:   for each unvisited point  $P$  in dataset  $D$  do
5:     Mark  $P$  as visited
6:      $N \leftarrow \text{Neighbors}(P, \epsilon, D)$   $\triangleright$  Find points within  $\epsilon$  distance
7:     if  $|N| \geq \text{MinPts}$  then
8:       Create a new cluster  $C_i$ 
9:       ExpandCluster( $P, N, C_i, \epsilon, \text{MinPts}, D$ )
10:      Add  $C_i$  to the list of clusters  $C$ 
11:     else
12:       Mark  $P$  as noise
13:     end if
14:   end for
15:   return  $C$   $\triangleright$  List of clusters
16: end procedure
17: procedure EXPANDCLUSTER( $P, N, C, \epsilon, \text{MinPts}, D$ )
18:   Add  $P$  to cluster  $C$ 
19:   for  $Q$  in  $N$  do
20:     if  $Q$  is not visited then
21:       Mark  $Q$  as visited
22:        $N' \leftarrow \text{Neighbors}(Q, \epsilon, D)$   $\triangleright$  Find neighbors of  $Q$ 
23:       if  $|N'| \geq \text{MinPts}$  then
24:         ExpandCluster( $Q, N', C, \epsilon, \text{MinPts}, D$ )
25:       end if
26:     end if
27:     if  $Q$  is not yet a member of any cluster then
28:       Add  $Q$  to cluster  $C$ 
29:     end if
30:   end for
31: end procedure

```

Here, we provide an overview of several commonly utilized unsupervised learning algorithms for clustering analysis. Clustering, a fundamental aspect of data analysis, aims to group similar data points together based on their inherent patterns or similarities. Unsupervised learning algorithms are pivotal in clustering tasks as they operate without the need for prior knowledge or labeled data. Within this section, we delve into four prominent unsupervised learning algorithms: DBSCAN, HDBSCAN, Mean Shift Clustering, and OPTICS. Algorithm 2 through 5 furnish detailed descriptions of these four unsupervised learning methods at hand, while Fig. 22 presents a comprehensive comparison outlining their key characteristics, advantages, and disadvantages. Mean Shift Clustering is a versatile non-parametric algorithm (see Algorithm 3) that identifies clusters by locating modes or peaks in the density distribution of data points. Through iterative shifts of data points towards higher density regions, it converges to cluster assignments. Its adaptability to various data shapes and sizes makes it suitable for datasets with irregular cluster structures. Ordering Points To Identify the Clustering Structure (OPTICS) is a density-based clustering algorithm (see Algorithm 5) that generates a data point ordering based on their density reachability. This approach captures clusters at different density levels, offering flexibility in cluster identification. Density-Based Spatial Clustering of Applications with Noise (DBSCAN) is another density-based algorithm (see Algorithm 2) that identifies

Algorithm	Key Characteristics	Pros	Cons
HDBSCAN	- Hierarchical Density-Based Clustering - Robust to varying density - Detects clusters of varying shapes - Memory-efficient (approximations exist)	- Produces hierarchical clustering structures - Handles noise and outliers well - No need to specify the number of clusters - Can discover clusters of different sizes	- Computational complexity grows with data size. - Sensitivity to parameter settings
OPTICS	- Orders points by density - Reveals hierarchical cluster structure - No need to specify the number of clusters	- Robust to varying density - Handles noise and outliers well - Can discover clusters of different sizes	- Memory-intensive for Large Datasets - Sensitivity to parameter Settings
Mean Shift Clustering	- Shifts data points toward mode - No need to specify the number of clusters - Adapts to varying density	- Automatically determines cluster centers - Robust to outliers - Handles irregularly shaped clusters	- Sensitive to bandwidth parameter - Computationally intensive for large datasets - Sensitivity to parameter settings
DBSCAN	- Density-Based Clustering - Detects clusters of varying shapes - No need to specify the number of clusters	- Robust to varying density - Identifies noise points and outliers - Suitable for datasets with irregular shapes	- Sensitive to epsilon and min_samples values - Not well-suited for high-dimensional data

Fig. 22. Comparison of unsupervised learning algorithms for clustering.

Algorithm 5 OPTICS (Ordering Points To Identify the Clustering Structure)

```

1: procedure OPTICS( $D, \epsilon, \text{MinPts}$ )
2:   Initialize an empty list of clusters  $C$ 
3:   Initialize an empty priority queue  $Q$  for seeds
4:   for each unvisited point  $P$  in dataset  $D$  do
5:     Mark  $P$  as visited
6:      $N \leftarrow \text{Neighbors}(P, \epsilon)$   $\triangleright$  Find points within  $\epsilon$  distance
7:     if  $|N| \geq \text{MinPts}$  then
8:        $P_{\text{core}} \leftarrow P$ 
9:       Create a new cluster  $C_i$ 
10:      Add  $P$  to cluster  $C_i$ 
11:       $N' \leftarrow \text{OrderNeighbors}(P_{\text{core}}, N, \epsilon)$   $\triangleright$  Order neighbors by
reachability distance
12:      ExpandCluster( $P_{\text{core}}, N', C_i, \epsilon, \text{MinPts}, Q$ )
13:      Add  $C_i$  to  $C$ 
14:    end if
15:  end for
16:  return  $C$   $\triangleright$  List of clusters
17: end procedure
18: procedure EXPANDCLUSTER( $P_{\text{core}}, N', C, \epsilon, \text{MinPts}, Q$ )
19:   for  $Q$  in  $N'$  do
20:     if  $Q$  is not visited then
21:       Mark  $Q$  as visited
22:        $N'' \leftarrow \text{Neighbors}(Q, \epsilon)$   $\triangleright$  Find points within  $\epsilon$  distance
23:       if  $|N''| \geq \text{MinPts}$  then
24:          $N' \leftarrow \text{OrderNeighbors}(P_{\text{core}}, N'', \epsilon)$   $\triangleright$  Order neighbors by
reachability distance
25:         ExpandCluster( $P_{\text{core}}, N', C, \epsilon, \text{MinPts}, Q$ )
26:       end if
27:     end if
28:   if  $Q$  is not in any cluster then
29:     Add  $Q$  to  $C$ 
30:   end if
31: end for
32: end procedure

```

clusters based on data point density. It groups densely connected points and distinguishes outliers as noise. DBSCAN excels in discovering clusters of arbitrary shapes and sizes, proving valuable in handling complex and irregular data distributions. Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN) extends DBSCAN by incorporating a hierarchical clustering approach (see Algorithm 4). It constructs a tree-based representation of clusters, facilitating robust handling of varying densities and complex cluster structures. HDBSCAN automatically determines the number of clusters and effectively captures both dense regions and outliers. OPTICS, on the other hand, is resilient to varying densities and proficient in handling data noise and outliers. These unsupervised learning algorithms provide valuable

tools for exploratory data analysis by revealing underlying structures, identifying cluster patterns, and facilitating insights into the inherent organization of the data.

References

- Akhtar, N., Mian, A., 2018. Threat of adversarial attacks on deep learning in computer vision: A survey. *Ieee Access* 6, 14410–14430.
- Alhajjar, E., Maxwell, P., Bastian, N., 2021. Adversarial machine learning in network intrusion detection systems. *Expert Syst. Appl.* 186, 115782.
- Altındış, F., Yılmaz, B., Borisenok, S., İçöz, K., 2021. Parameter investigation of topological data analysis for EEG signals. *Biomed. Signal Process. Control* 63, 102196.
- Azmoodeh, A., Dehghantanha, A., Choo, K.-K.R., 2018. Robust malware detection for internet of (battlefield) things devices using deep eigenspace learning. *IEEE Trans. Sustain. Comput.* 4 (1), 88–95.
- Barreno, M., Nelson, B., Joseph, A.D., Tygar, J.D., 2010. The security of machine learning. *Mach. Learn.* 81, 121–148.
- Barreno, M., Nelson, B., Sears, R., Joseph, A.D., Tygar, J.D., 2006. Can machine learning be secure? In: *Proceedings of the 2006 ACM Symposium on Information, Computer and Communications Security*. pp. 16–25.
- Bhagoji, A.N., Cullina, D., Mittal, P., 2017. Dimensionality reduction as a defense against evasion attacks on machine learning classifiers. 2, 1, arXiv preprint arXiv:1704.02654.
- Biggio, B., Roli, F., 2018a. Wild patterns: Ten years after the rise of adversarial machine learning. In: *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. pp. 2154–2156.
- Biggio, B., Roli, F., 2018b. Wild patterns: Ten years after the rise of adversarial machine learning. *Pattern Recognit.* 84, 317–331.
- Campigotto, R., Poon, H., 2019. A scalable kernel k-means algorithm with topological data analysis. *Pattern Recognit.* 88, 180–195.
- Carlsson, G., 2009. Topology and data. *Bull. Amer. Math. Soc.* 46 (2), 255–308.
- Chakraborty, A., Alam, M., Dey, V., Chattopadhyay, A., Mukhopadhyay, D., 2018. Adversarial attacks and defenses: A survey. arXiv preprint arXiv:1810.00069.
- Chale, M., Cox, B., Weir, J., Bastian, N., 2023. Constrained optimization based adversarial example generation for transfer attacks in network intrusion detection systems. *Springer Optim. Lett.*
- Chen, Y., Lloyd, W., 2020. Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science. *Sci. Rep.* 10 (1), 1–13.
- Dalvi, N., Domingos, P., Sanghai, S., Verma, D., 2004. Adversarial classification. In: *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. pp. 99–108.
- De Lucia, M.J., Cotton, C., 2019. Adversarial machine learning for cyber security. *J. Inform. Syst. Appl. Res.* 12 (1), 26.
- Devine, S.M., Bastian, N.D., 2021. An adversarial training based machine learning approach to malware classification under adversarial conditions. In: *HICSS*. pp. 1–10.
- Farrukh, Y.A., Khan, I., Wali, S., Bierbrauer, D., Pavlik, J.A., Bastian, N.D., 2022. Payload-byte: A tool for extracting and labeling packet capture files of modern network intrusion detection datasets. In: *2022 IEEE/ACM International Conference on Big Data Computing, Applications and Technologies. BDCAT, IEEE*, pp. 58–67.
- Feinman, R., Curtin, R.R., Shintre, S., Gardner, A.B., 2017. Detecting adversarial samples from artifacts. arXiv preprint arXiv:1703.00410.
- Gao, Y., Xu, C., Wang, D., Chen, S., Ranasinghe, D.C., Nepal, S., 2019a. Strip: A defence against trojan attacks on deep neural networks. In: *Proceedings of the 35th Annual Computer Security Applications Conference*. pp. 113–125.
- Gao, Y., et al., 2019b. A model-based anomaly detection method for machine learning systems. *IEEE Trans. Dependable Secure Comput.* 16 (4), 563–576.

- Giansiracusa, N., Giansiracusa, R., Moon, C., 2019. Persistent homology machine learning for fingerprint classification. In: 2019 18th IEEE International Conference on Machine Learning and Applications. ICMLA, IEEE, pp. 1219–1226.
- Gong, Z., Wang, W., Ku, W.-S., 2017. Adversarial and clean data are not twins. arXiv preprint arXiv:1704.04960.
- Goodfellow, I., McDaniel, P., Papernot, N., 2018. Making machine learning robust against adversarial inputs. *Commun. ACM* 61 (7), 56–66.
- Grosse, K., Manoharan, P., Papernot, N., Backes, M., McDaniel, P., 2017. On the (statistical) detection of adversarial examples. arXiv preprint arXiv:1702.06280.
- Guo, W., Manohar, K., Brunton, S.L., Banerjee, A.G., 2018. Sparse-TDA: Sparse realization of topological data analysis for multi-way classification. *IEEE Trans. Knowl. Data Eng.* 30 (7), 1403–1408.
- Guo, C., Rana, M.A., Shen, C., Wu, C., 2020. A review on intrusion detection systems using machine learning techniques. *J. Netw. Comput. Appl.* 149, 102461.
- Guo, W., Tondi, B., Barni, M., 2022. An overview of backdoor attacks against deep neural networks and possible defences. *IEEE Open J. Signal Process.*
- Hofer, C., Kwitt, R., Niethammer, M., 2017. Deep learning with topological signatures. In: *Advances in Neural Information Processing Systems*. pp. 163–172.
- Hore, S., Ghadermazi, J., Paudel, D., Shah, A., Das, T.K., Bastian, N.D., 2023. Deep PackGen: A deep reinforcement learning framework for adversarial network packet generation. arXiv:2305.11039.
- Huang, L., Joseph, A.D., Nelson, B., Rubinstein, B.I., Tygar, J.D., 2011. Adversarial machine learning. In: *Proceedings of the 4th ACM Workshop on Security and Artificial Intelligence*. pp. 43–58.
- Kolias, C., Kambourakis, G., Stavrou, A., Voas, J., 2017. Intrusion detection in 21st century: A state-of-the-art review. *IEEE Commun. Surv. Tutor.* 19 (2), 1801–1821.
- Kuznetsov, P., Edmunds, R., Xiao, T., Iqbal, H., Puri, R., Golmant, N., Shih, S., 2018. Adversarial machine learning. In: *Artificial Intelligence Safety and Security*. Chapman and Hall/CRC, pp. 235–248.
- Li, X., Li, W., Zhou, J., Guo, P., Chen, Y., 2023. Hybrid deep learning framework for fake image detection using visual content and metadata analysis. *Knowl.-Based Syst.* 237, 107088.
- Lin, F.Y.-S., Wang, Y.-S., Huang, M.-Y., 2013. Effective proactive and reactive defense strategies against malicious attacks in a virtualized honeynet. *J. Appl. Math.* 2013.
- Liu, Q., Li, P., Zhao, W., Cai, W., Yu, S., Leung, V.C., 2018. A survey on security threats and defensive techniques of machine learning: A data driven view. *IEEE Access* 6, 12103–12117.
- Metzen, J.H., Genewein, T., Fischer, V., Bischoff, B., 2017. On detecting adversarial perturbations. arXiv preprint arXiv:1702.04267.
- Milnor, J., 1963. *Morse Theory*. Princeton University Press.
- Monkam, G.F., De Lucia, M.J., Bastian, N.D., 2023. Preprocessing network traffic using TDA for data poisoning detection. In: *Proceedings of the 2023 IEEE Conference on Dependable and Secure Computing*. IEEE.
- Munir, S., Hafeez, K., Abbas, H., Kwak, K.S., Kim, B.S., 2020. A review of data poisoning attacks on machine learning systems. *Neurocomputing* 396, 399–415.
- Nicolae, M.M., Sinn, M., Gehr, T., Tsankov, P., Vechev, M., 2018. Formal guarantees on the robustness of a classifier against adversarial manipulation. In: *Advances in Neural Information Processing Systems*. pp. 2266–2277.
- Papernot, N., McDaniel, P., Jha, S., Fredrikson, M., Celik, Z.B., Swami, A., 2016. The limitations of deep learning in adversarial settings. In: 2016 IEEE European Symposium on Security and Privacy. EuroS&P, IEEE, pp. 372–387.
- Papernot, N., McDaniel, P., Jha, S., Fredrikson, M., Celik, Z.B., Swami, A., 2018a. The limitations of deep learning in adversarial settings. In: *IEEE European Symposium on Security and Privacy*. EuroS&P, pp. 372–387.
- Papernot, N., McDaniel, P., Sinha, A., Wellman, M.P., 2018b. Sok: Security and privacy in machine learning. In: 2018 IEEE European Symposium on Security and Privacy. EuroS&P, IEEE, pp. 399–414.
- Papernot, N., McDaniel, P., Sinha, A., Wellman, M., Goodfellow, I., 2017. Practical black-box attacks against machine learning. In: *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*. ACM, pp. 506–519.
- Qi, F., Chen, Y., Li, M., Yao, Y., Liu, Z., Sun, M., 2020. Onion: A simple and effective defense against textual backdoor attacks. arXiv preprint arXiv:2011.10369.
- Reininghaus, J., Huber, S., Bauer, U., Kwitt, R., Richter-Gebert, J., Jähne, B., 2014. A stable multi-scale kernel for topological machine learning. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. pp. 4741–4748.
- Ribeiro, M.T., Singh, S., Guestrin, C., 2016. “Why should I trust you?” explaining the predictions of any classifier. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. pp. 1135–1144.
- Romano, D., Nicolau, M., Quintin, E.-M., Mazaika, P.K., Lightbody, A.A., Cody Hazlett, H., Piven, J., Carlsson, G., Reiss, A.L., 2014. Topological methods reveal high and low functioning neuro-phenotypes within fragile X syndrome. *Hum. Brain Map.* 35 (9), 4904–4915.
- Rucco, M., Castiglione, F., Merelli, E., Pettini, M., 2016. Characterisation of the idiotypic immune network through persistent entropy. In: *Proceedings of ECCS 2014: European Conference on Complex Systems*. Springer, pp. 117–128.
- Severi, G., Meyer, J., Coull, S.E., Oprea, A., 2021. Explanation-guided backdoor poisoning attacks against malware classifiers. In: *USENIX Security Symposium*. pp. 1487–1504.
- Shipp, T.J., Clouse, D.J., De Lucia, M.J., Ahiskali, M.B., Stevenson, K., Mullin, J.M., Bastian, N.D., 2020. Advancing the research and development of assured artificial intelligence and machine learning capabilities. arXiv preprint arXiv:2009.13250.
- Shu, K., et al., 2022. A comprehensive survey on poisoning attacks and defenses in federated learning. *Future Gener. Comput. Syst.* 128, 202–220.
- Singh, G., Memoli, F., Carlsson, G., 2007. Topological methods for the analysis of high dimensional data sets and 3D object recognition. In: *SPBG*, vol. 7, (no. 8), Citeseer, pp. 91–100.
- Sizemore, A.E., Phillips-Cremins, J.E., Ghrist, R., Bassett, D.S., 2019. The importance of the whole: Topological data analysis for the network neuroscientist. *Netw. Neurosci.* 3 (3), 656–673.
- Smith, J., et al., 2020. Detecting and mitigating data poisoning attacks in AI models. *J. Artificial Intelligence Res.* 67, 675–712.
- Songdechakraiwt, T., Chung, M.K., 2020. Dynamic topological data analysis for functional brain signals. In: 2020 IEEE 17th International Symposium on Biomedical Imaging Workshops. ISBI Workshops, IEEE, pp. 1–4.
- Suri, N., Tortonesi, M., Michaelis, J., Budulas, P., Benincasa, G., Russell, S., Stefanelli, C., Winkler, R., 2016. Analyzing the applicability of internet of things to the battlefield environment. In: 2016 International Conference on Military Communications and Information Systems. ICMCIS, IEEE, pp. 1–8.
- Tabassi, E., Burns, K.J., Hadjimichael, M., Molina-Markham, A.D., Sexton, J.T., 2019. A taxonomy and terminology of adversarial machine learning. *NIST IR* 2019, 1–29.
- Talty, K., Stockdale, J., Bastian, N.D., 2021. A sensitivity analysis of poisoning and evasion attacks in network intrusion detection system machine learning models. In: *MILCOM 2021-2021 IEEE Military Communications Conference*. MILCOM, IEEE, pp. 1011–1016.
- Tauzin, G., Lupo, U., Tunstall, L., Pérez, J.B., Caorsi, M., Medina-Mardones, A.M., Dassatti, A., Hess, K., 2021. giotto-tda: A topological data analysis toolkit for machine learning and data exploration. *J. Mach. Learn. Res.* 22 (39), 1–6, URL <http://jmlr.org/papers/v22/20-325.html>.
- Venkatesan, S., Sikka, H., Izmailov, R., Chadha, R., Oprea, A., de Lucia, M.J., 2021. Poisoning attacks and data sanitization mitigations for machine learning models in network intrusion detection systems. In: *MILCOM 2021 - 2021 IEEE Military Communications Conference*. MILCOM, pp. 874–879. <http://dx.doi.org/10.1109/MILCOM52596.2021.9652916>.
- Wang, Z., Mei, K., Ding, H., Zhai, J., Ma, S., 2022. Rethinking the reverse-engineering of Trojan Triggers. arXiv preprint arXiv:2210.15127.
- Yuan, X., He, P., Zhu, Q., Li, X., 2019. Adversarial examples: Attacks and defenses for deep learning. *IEEE Trans. Neural Netw. Learn. Syst.* 30 (9), 2805–2824.

Dr. Galamo F. Monkam is a Postdoctoral Researcher within the Data and Decision Sciences Division at the Army Cyber Institute, United States Military Academy.

Dr. Michael J. De Lucia is a Computer Scientist within the Network Security Branch at the Army Research Laboratory, U.S. Army Combat Capabilities Development Command.

Dr. Nathaniel D. Bastian is Division Chief, Data and Decision Sciences and Senior Research Scientist at the Army Cyber Institute, United States Military Academy.